UNIT-5

Machine Translation:

5	**Machine Translation:** Machine Translation (Seq2Seq), Paradigm Shift in NLP (BERT, LaMbDa, GPT), NLP in AI (Conversational AI etc.), Understand and discuss few current research papers related to NLP.

THE PROBLEM

Why Not a Regular Neural Network?

- Traditional feed-forward neural networks are designed to work with fixed-size inputs and outputs.
- Even a basic RNN-based classifier generally produces a **fixed type or fixed-size output.**
- However, natural language is variable in length.
- A sentence may contain 3 words, 5 words, or 20 words, and the required output may have a **completely different length.**
- This creates a challenge for language-generation tasks such as translation, summarization, and dialogue generation.

Fixed-Size I/O — The Traditional Constraint

Consider a simple classification-style setup:

Input:
I love NLP → 3 words

If the model is designed for a fixed-size output, it cannot naturally generate an output sequence of arbitrary length.

For example, in machine translation:

English:

`I am learning NLP` → **4 words**

Hindi:

`मैं NLP सीख रहा हूँ` → **4 words**

But consider:

English:

`I am learning Natural Language Processing` → **6 words**

Hindi:

`मैं प्राकृतिक भाषा संसाधन सीख रहा हूँ` → **6+ words**

The input and output lengths are not necessarily the same.

A conventional fixed-size architecture does not naturally handle this **variable-length → variable-length** problem.

Real-Life Example: Text Summarization

Suppose we give the model a long sentence:

Input — 15 words:

> **Artificial intelligence is transforming healthcare by helping doctors detect diseases faster and make better clinical decisions.**

The model can generate a much shorter summary:

Output — 6 words:

> **AI helps doctors detect diseases faster.**

Notice the difference:

15 words → 6 words

A fixed-size architecture would struggle to naturally handle this changing output length.

A Seq2Seq model can:

> Read an input of any reasonable length → understand its meaning → generate an output of the required length.

Solution : Sequence-to-Sequence Models

Basic Idea

A sequence is simply a set of items arranged in order.

For example:

`I → love → NLP`

Here, the words appear in a particular order, so they form a sequence.

What Does Sequence-to-Sequence Mean?

A Sequence-to-Sequence (Seq2Seq) model takes one sequence as input and produces another sequence as output.

Input Sequence → Seq2Seq Model → Output Sequence

Example:

I love NLP → Model → मुझे NLP पसंद है

The input and output can have **different lengths.**

Why Do We Need Seq2Seq Models?

Consider machine translation.

English:

I am going to college.

Hindi:

मैं कॉलेज जा रही हूँ।

The number of words may be different, but the meaning needs to be preserved.

Therefore, we need a model that can:

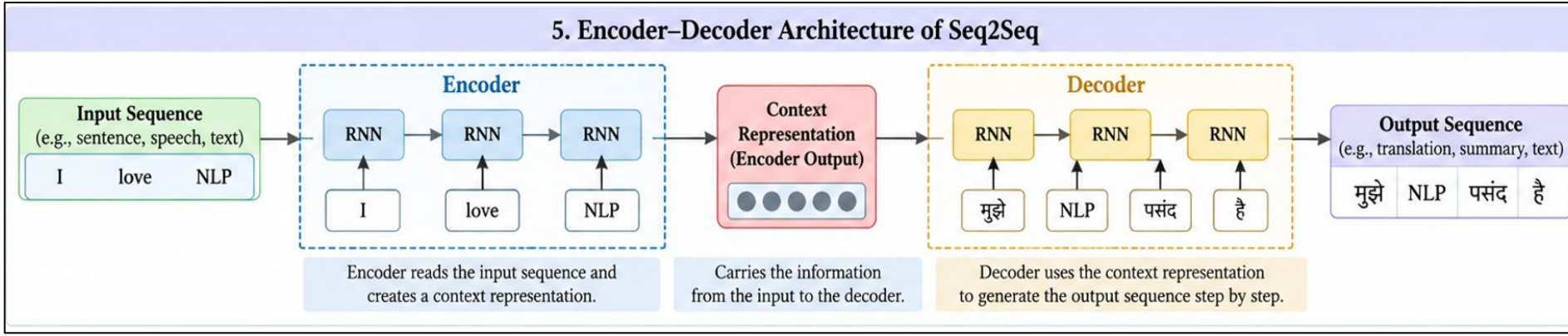
Understand one sequence → Generate another sequence

This is where Seq2Seq models are useful.

The Main Architecture: Encoder–Decoder

A Seq2Seq model mainly consists of two parts:

Encoder + Decoder



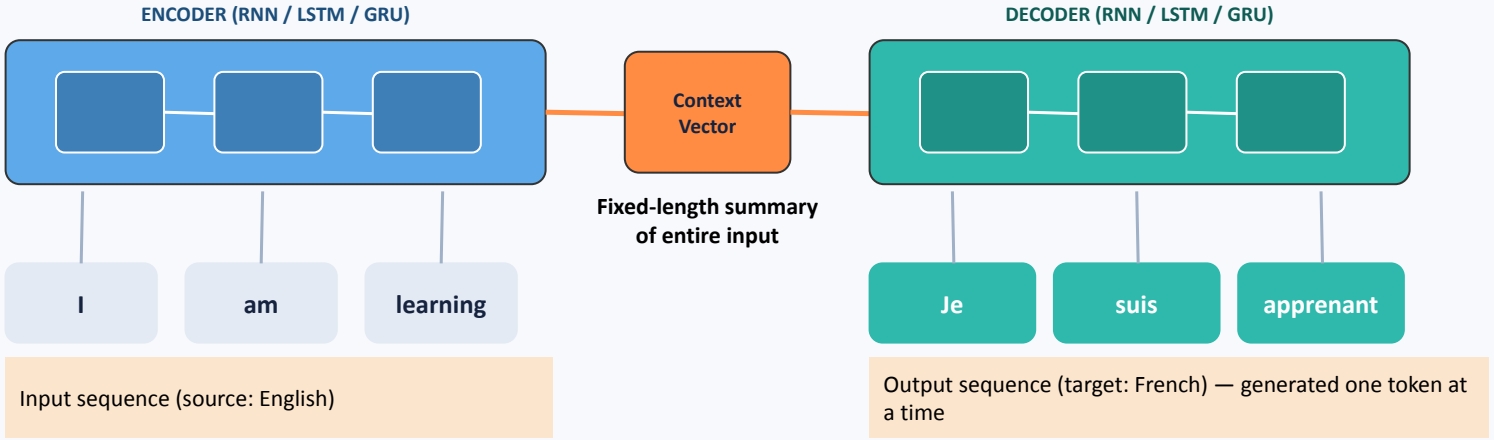
Think of it like a translator:

Encoder = Understand the sentence

Decoder = Generate the translated sentence

Encoder → Context Vector → Decoder

Two RNNs joined by a single vector: one reads, one writes. This is the anatomy every Seq2Seq model shares **(English to French).**



Encoder

The Encoder reads the sentence word by word and understands the information step by step.

Context Vector

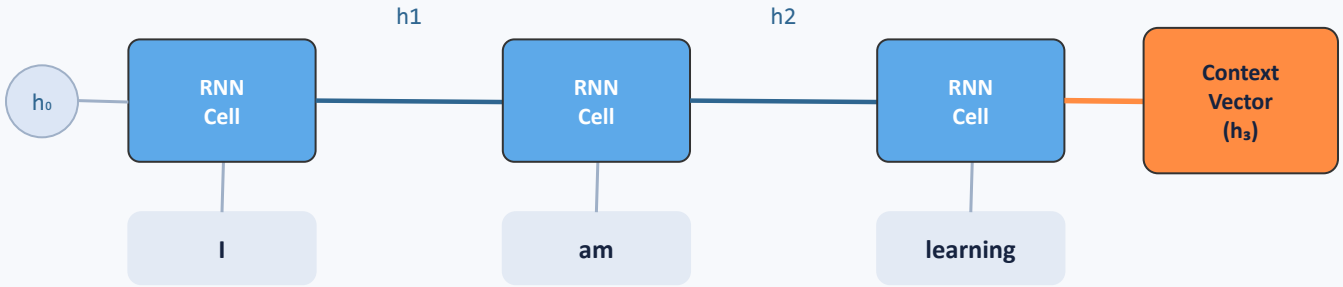
After reading the sentence, the Encoder creates a single numerical representation called the context vector. It carries the important information or meaning of the input sentence

Decoder

The Decoder takes the context vector and generates the output one word at a time.

Inside the Encoder

The encoder is an RNN/LSTM/GRU that never produces visible output — its only job is to build up a hidden state that remembers the sentence.



Hidden state h□ is passed forward at every step, accumulating context as each new word arrives.

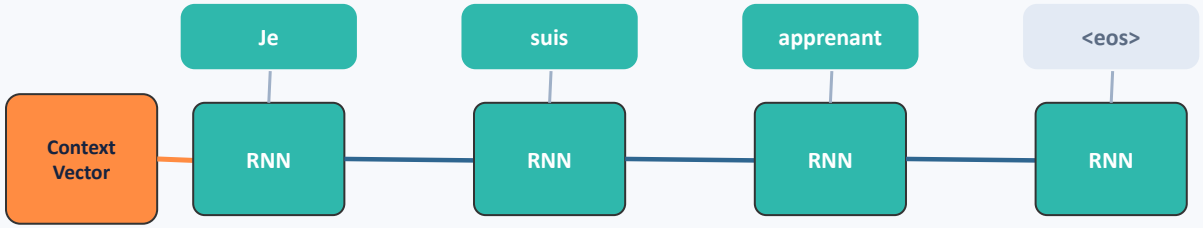
The Recurrence Equation

$$h_t = \sigma(W_{hx} \cdot x_t + W_{hh} \cdot h_{t-1})$$

x_t	**Current input** — the word/token being processed now
h_{t-1}	**Previous memory** — information remembered from the previous word
W_{hx}	Learned weights that determine how much importance to give the current input
W_{hh}	Learned weights that determine how much information to retain from previous memory
σ	Activation function, usually **tanh**, that controls the resulting values
h_t	**New memory** after processing the current input

Inside the Decoder

The decoder is also an RNN/LSTM/GRU — but it runs in reverse: given a state, it must produce words, one at a time, feeding each prediction back in.



Hidden state passes forward through each step; the token predicted at step t feeds back in as the input for step t+1 (inference mode).

How each token is chosen

- Start with a special <sos> token
- RNN cell outputs a score for every word in the vocabulary
- Softmax turns scores into probabilities
- Highest-probability word is emitted
- Generation stops at <eos> or a max length

The Output Equation

$$\hat{y}_t = \text{softmax}(W_{yh} \cdot h_t)$$

Wyh — Learned Weights

The hidden state is passed through learned weights:

This produces a score for every word in the vocabulary.

Hidden State ht

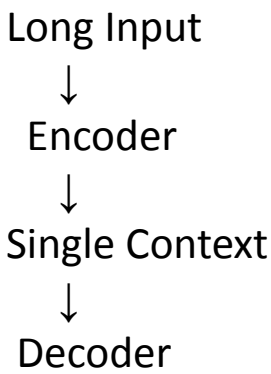
The decoder's hidden state contains the information it currently knows:

Where Are Seq2Seq Models Used?

Application	Input	Output
Machine Translation	English sentence	Hindi sentence
Text Summarization	Long document	Short summary
Speech Recognition	Speech	Text
Chatbots	User question	Response
Question Answering	Question	Answer

What Was the Problem with the Basic Seq2Seq Model?

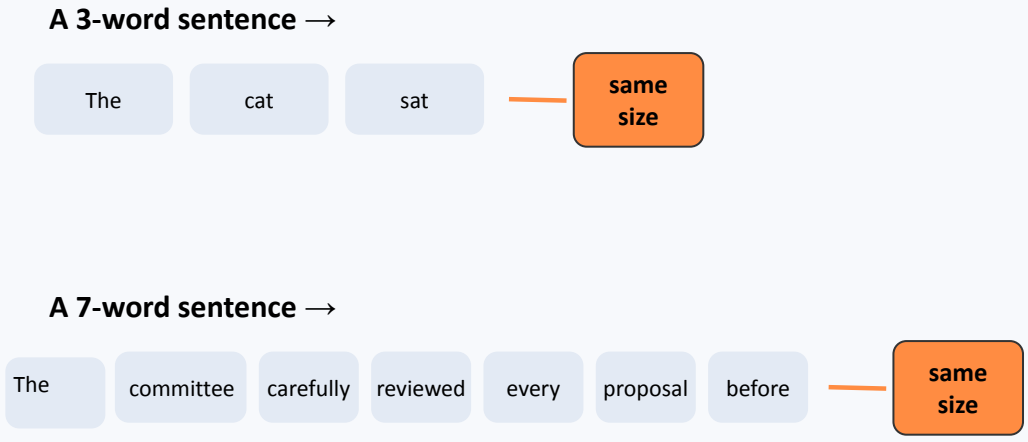
- Suppose the input is a very long sentence or paragraph.
- The Encoder tries to represent the entire input using a limited context representation.
- This can create an information bottleneck.



Some important information may be difficult to retain.

The Fixed-Length Bottleneck

Squeezing an entire sentence — 5 words or 50 — into one vector of the same size is the model's biggest weakness.

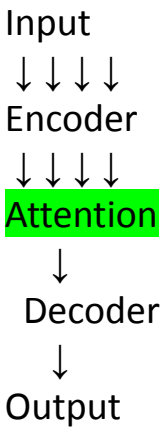


What goes wrong

- Early words get diluted by the time the last word is encoded
- Long sentences lose fine-grained detail — translation quality drops sharply past ~20-30 words
- The decoder has no way to look back at specific input words — it only sees one blended summary

Solution: Attention Mechanism

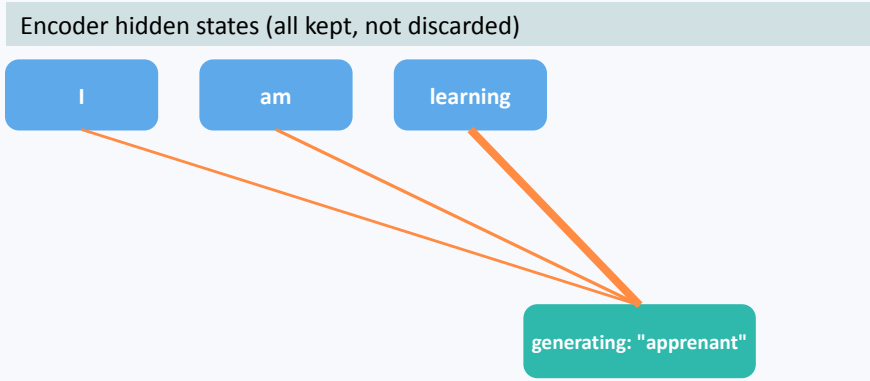
- Instead of making the Decoder depend only on one fixed representation, A**ttention** allows it to focus on the relevant parts of the input while generating each output.



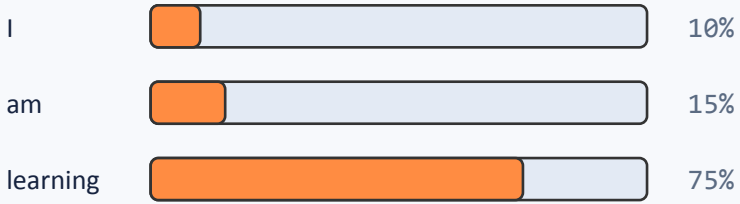
For example, when generating a particular translated word, the Decoder can pay more attention to the relevant input words.

Attention: Let the Decoder Look Back

Instead of one static context vector, attention builds a fresh, weighted context at every decoding step — looking at all encoder states, not just the last one.



Line thickness = attention weight — "learning" matters most for producing "apprenant"



Attention in three steps

Score

Compare the decoder's current state against every encoder hidden state to get relevance scores.

Weight

Turn scores into weights with softmax — they sum to 1 across all input positions.

Blend

Take a weighted sum of encoder states using those weights → a fresh context vector for this step.

Attention allows the model to look at all words in the input and automatically decide how important each word is for the current task. This idea became the foundation of the Transformer architecture.

Self-Attention

This attention idea was developed further into Self-Attention, which became a central mechanism of Transformers.

```
Attention
    ↓
Look at all input positions
    ↓
Calculate importance of each
    ↓
Give more weight to relevant words
    ↓
Create focused representation
    ↓
Self-Attention
    ↓
Transformer
```

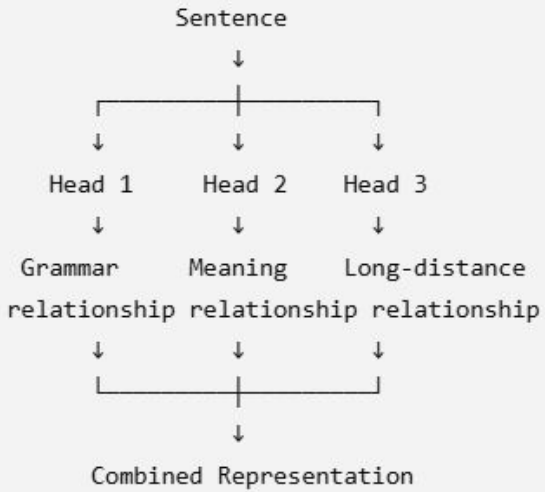
Multi-Head Attention

Example

Consider:

> **"The cat sat on the mat because it was tired."**

Different attention heads may focus on different relationships:



"Multi-head attention captures multiple relations at once" means:

- Instead of looking at a sentence in only one way,
- the Transformer uses multiple attention heads,
- and each head can focus on a different type of relationship between words.

Analogy: (Self Attention and multihead attention)

- Imagine you're reading a sentence.
- Self-attention would be like considering how each word relates to every other word to understand the overall meaning.
- Multi-head self-attention would be like reading the sentence several times, each time focusing on a different aspect like grammar, word relationships, or sentiment.
- By combining these focused readings, you get a richer understanding of the sentence.

Paradigm

A general modeling framework (a set of methods) used to solve a whole class of tasks, not just one dataset.

Example

Sequence-to-Sequence (Seq2Seq) is a standard paradigm for tasks like machine translation, summarization, etc.

Paradigm Shift

Solving a task using a different paradigm than the one it is traditionally solved with, by reformulating the task itself.

Example

Same dataset, new input/output shape — then a model built for that new shape solves it, often better.

Worked Example: Machine Translation

Traditional Approach: Sequence-to-Sequence (Seq2Seq)

Translate the input sentence into the target language.

Input (English Sentence)
I love NLP.

→ **Seq2Seq Model** (Encoder – Decoder) →

Output (Hindi Sentence)
मुझे NLP पसंद है।

Same task but different paradigm

New Approach: Instruction-based Text Generation

Reformulate translation as a text generation task using a prompt.

Input (Prompt + Sentence)

Instruction:
Translate the following English sentence into Hindi.

English Sentence:
I love NLP.

→ **Large Language Model** (Generative Model) →

Output (Hindi Sentence)
मुझे NLP पसंद है।

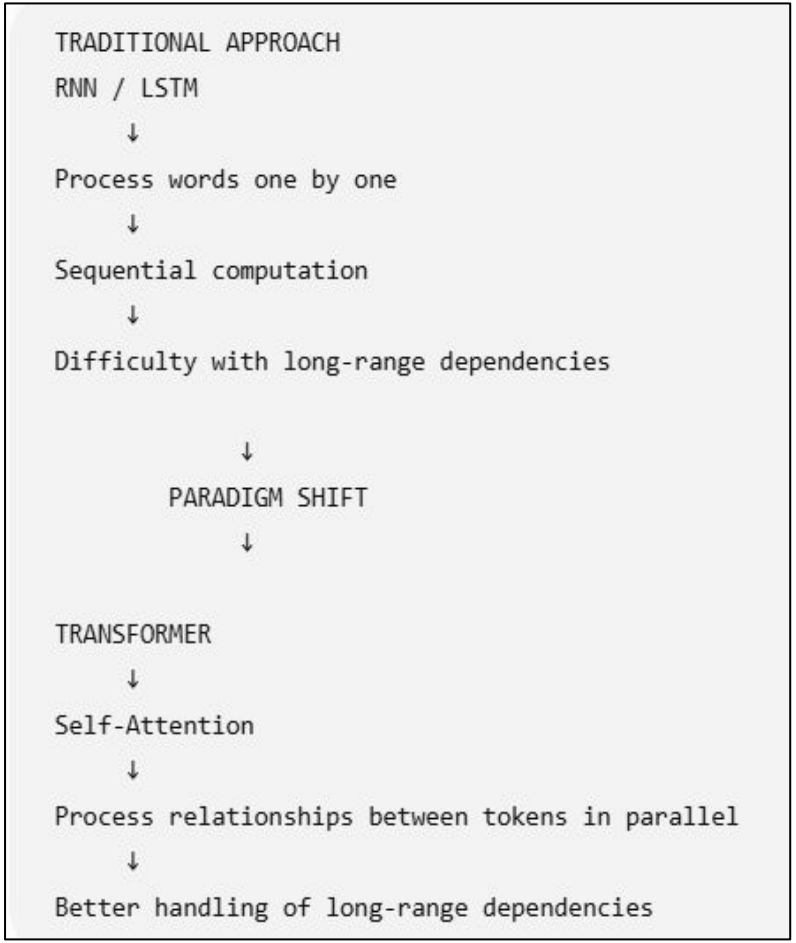
Result: Same dataset, same task (Machine Translation) — new input/output shape and a different modeling paradigm.

The Seven Mainstream NLP Paradigms

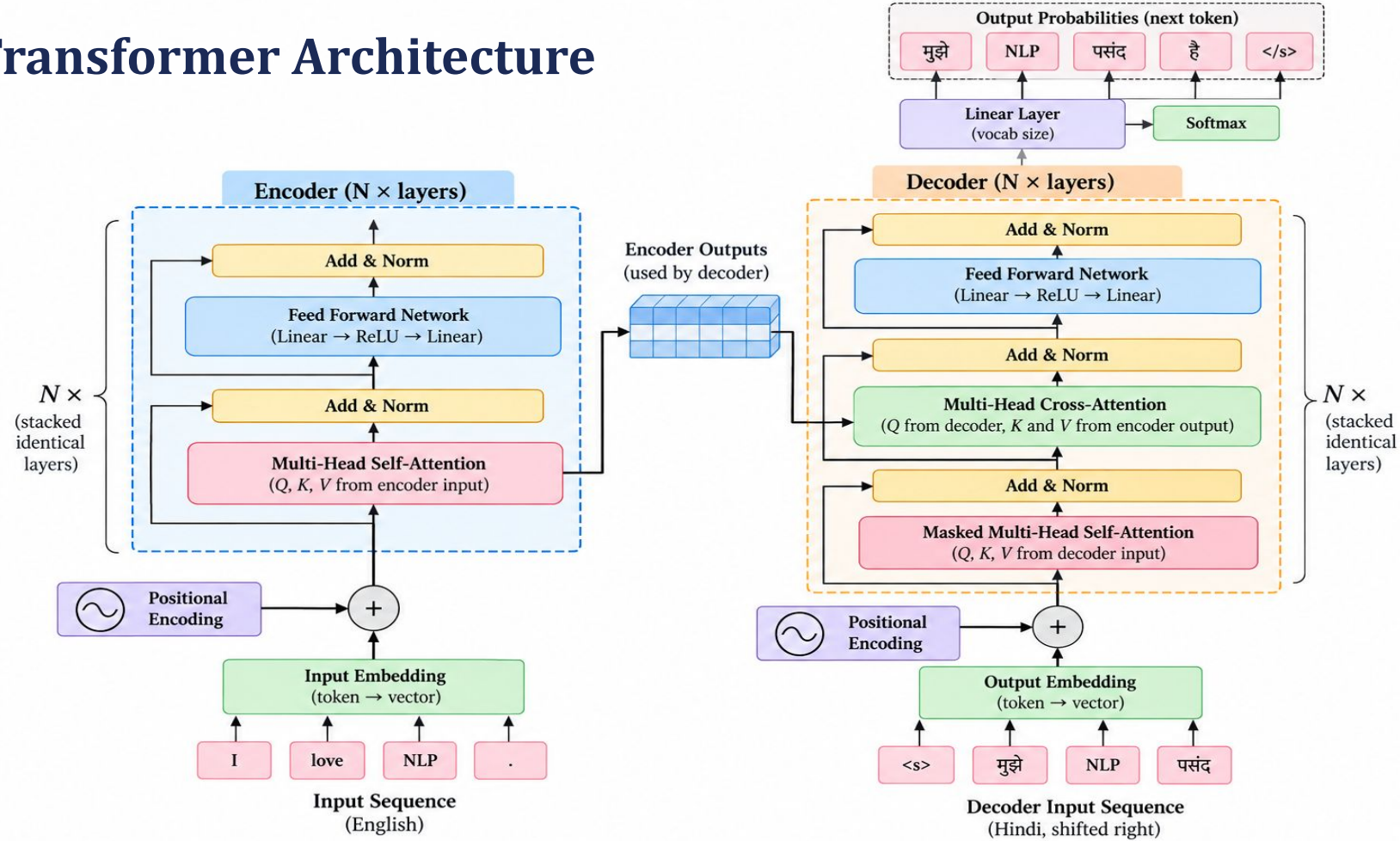
Paradigm	Basic Structure	What It Does	Typical Applications
Class	Encoder → Classifier → Label	Assigns one or more labels to the input	Sentiment analysis, topic classification
Matching	Encode two texts → Relate them	Determines the relationship or similarity between two texts	Natural Language Inference, information retrieval
SeqLab	Tag every token in a sequence	Assigns a label to each word/token	POS tagging, NER, chunking
MRC	Question + Text → Answer Span	Finds the relevant answer in a text based on a question	Reading comprehension, reformulated NER
Seq2Seq	Encoder → Decoder → New Sequence	Converts one sequence into another sequence	Machine translation, summarization
Seq2ASeq	Input → Action Sequence	Predicts a sequence of structured actions	Dependency parsing, constituency parsing
(M)LM	Prompt → Predict/Fill Tokens	Predicts missing or next tokens based on context	Few-shot/zero-shot prompting, GPT-style tuning

Transformer: a paradigm shift

RNN-based approach	Transformer approach
Sequential processing	Highly parallel processing
Recurrence is central	Self-attention is central
Information passes step by step	Tokens can directly attend to other tokens
Long-range dependencies can be difficult	Direct attention can connect distant tokens



Transformer Architecture



Step 1: Input Sentence

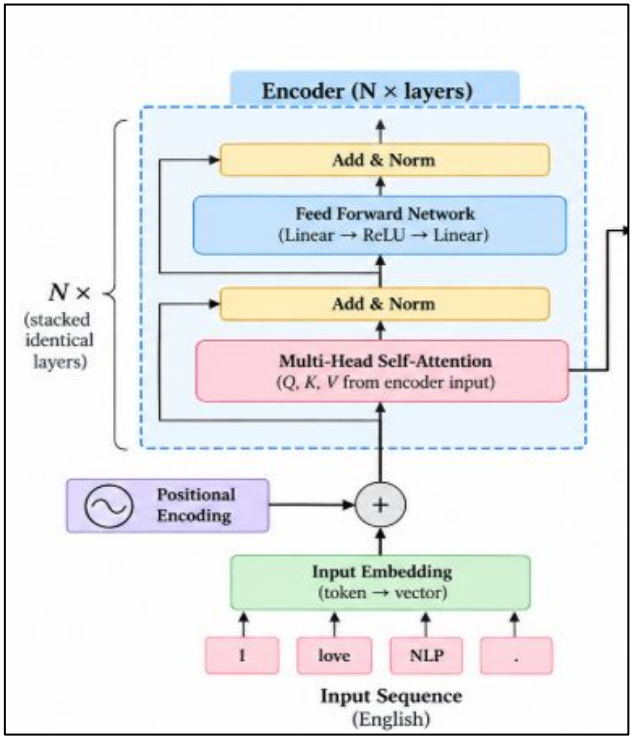
Suppose our input is:

I love NLP.

The sentence is divided into individual tokens:

I love NLP .

Each token is processed by the Transformer.



Step 2: Input Embedding

Computers cannot directly understand words such as “I”, “love”, or “NLP.”

So each token is converted into a numerical vector.

I → [0.2, 0.7, ...]
love → [0.5, 0.1, ...]
NLP → [0.8, 0.4, ...]

This is called an embedding.

Embedding = Convert words/tokens into numerical vectors.

Step 3: Positional Encoding

There is an important problem.

The Transformer processes tokens in parallel, so it needs information about word order.

For example:

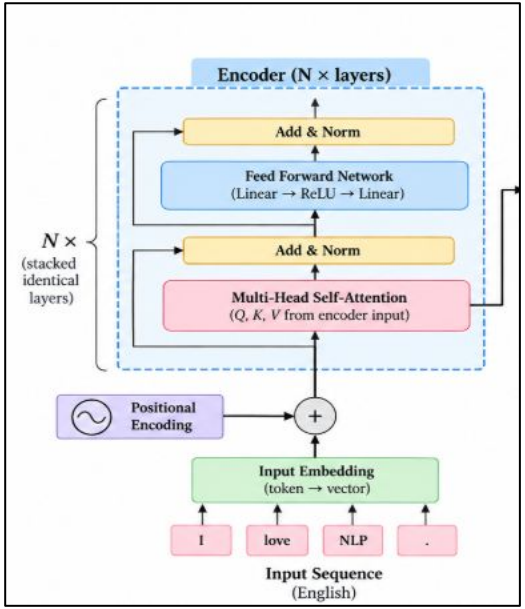
I love NLP

is different from:

NLP love I

Positional encoding tells the model:

I → Position 1
love → Position 2
NLP → Position 3



So:

Positional Encoding = Tells the Transformer where each word occurs in the sentence.

ENCODER

Step 4: Multi-Head Self-Attention

This is one of the most important parts of the Transformer.

The model looks at the words in relation to one another.

For example:

> “The animal didn't cross the road because it was tired.”

What does “it” refer to?

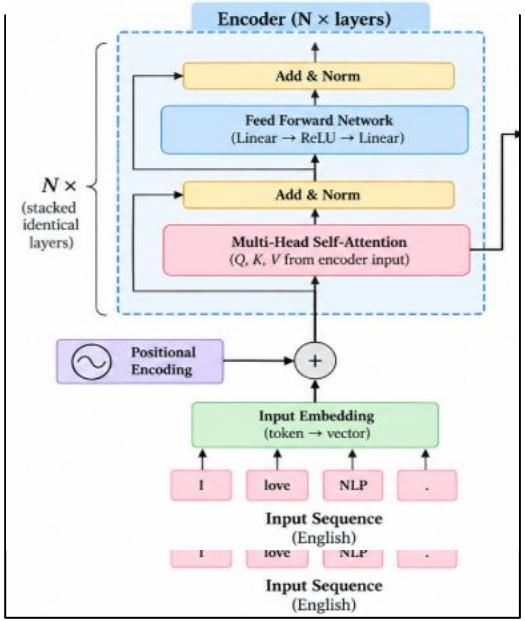
The model needs to understand the relationship between “**it” and “animal.”**

Self-attention allows every token to look at other tokens and determine:

> “Which other words are important for understanding me?”

The Encoder uses:

Query (Q) + Key (K) + Value (V) to calculate attention.

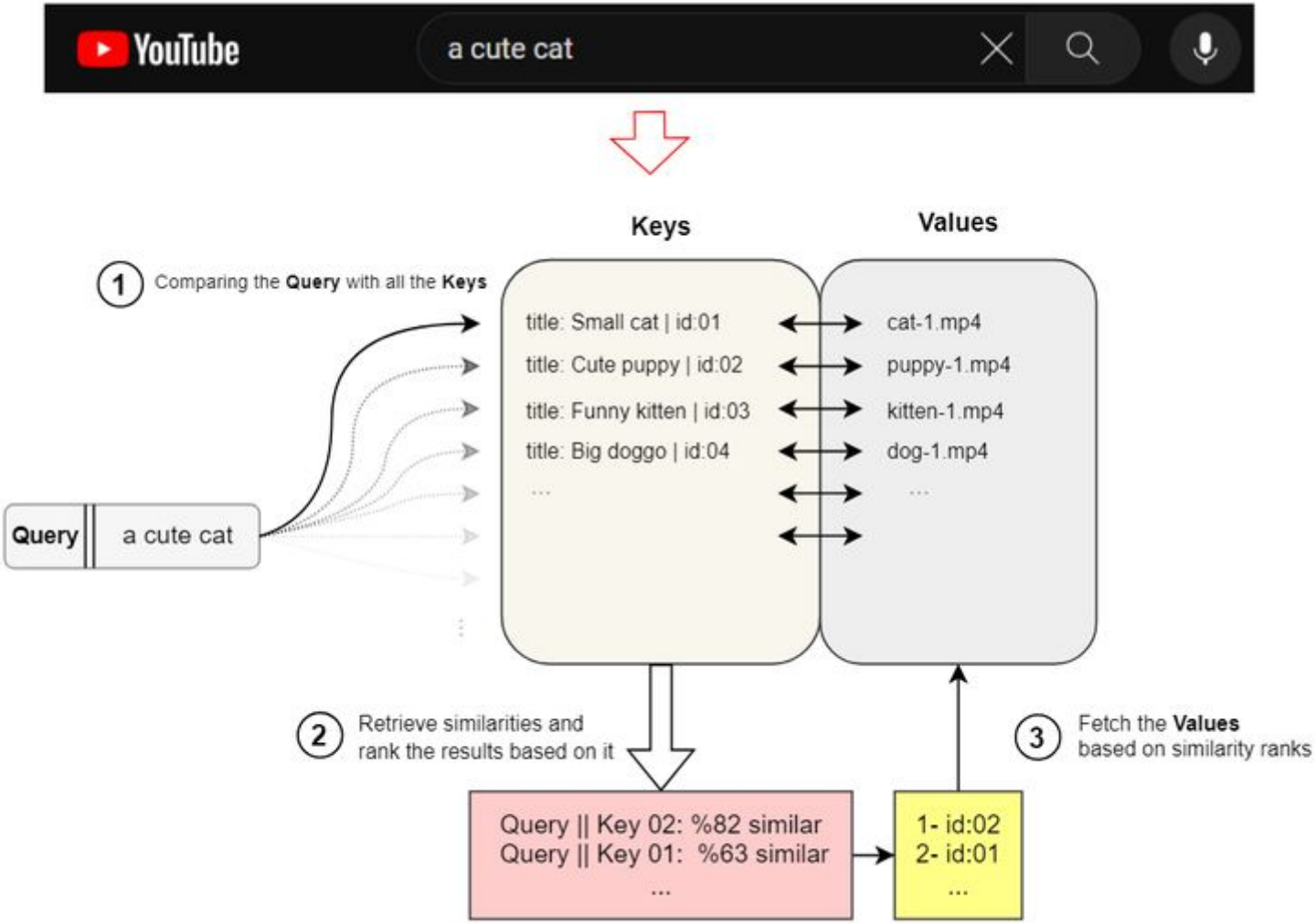


Query, *Key*, and *Value*

To make more sense, think of when you search for something on YouTube, for example. Assume YouTube stores all its videos as a pair of "*video title*" and the "*video file*" itself. Which we call a **Key-Value pair**, with the Key being the video title and the Value being the video itself.

The text you put in the search box is called a **Query** in search terms. So in a sense, when you search for something, **YouTube compares your search Query with the Keys of all its videos**, then measures the similarity between them, and ranks their **Values** from the highest similarity down.

https://towardsdatascience.com/what-are-query-key-and-value-in-the-transformer-architecture-and-why-are-they-used-acbe73f731f2/


YouTube
a cute cat
Keys
Values
1
Comparing the Query with all the Keys
title: Small cat | id:01
title: Cute puppy | id:02
title: Funny kitten | id:03
title: Big doggo | id:04
...
cat-1.mp4
puppy-1.mp4
kitten-1.mp4
dog-1.mp4
...
Query
a cute cat
2
Retrieve similarities and
rank the results based on it
3
Fetch the Values
based on similarity ranks
Query || Key 02: %82 similar
Query || Key 01: %63 similar
...
1- id:02
2- id:01
...

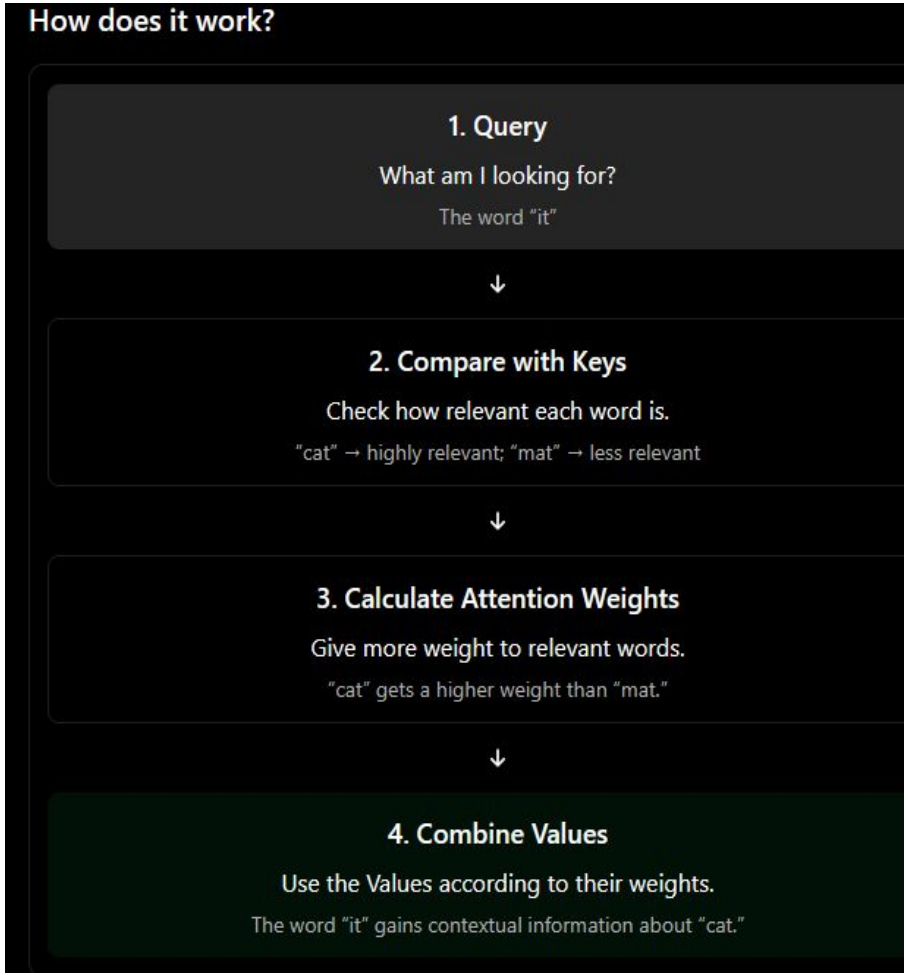
Attention in Transformers works like searching for the most relevant words in a sentence.

Let's understand it with an example.

Sentence: “The cat sat on the mat because it was tired.”

Suppose we want to understand the word “it.”

- **Query (Q):** What information am I looking for? The word “it” wants to find out which word it refers to.
- **Keys (K):** What information is available to search through? Each word in the sentence has a Key that helps determine how relevant it is to “it.”
- **Values (V):** What information can I take from each word? Each word has a Value containing information that can contribute to understanding “it.”



Imagine searching for a book in a library:

- **Query =** What book am I looking for?
- **Key** = The information used to identify and match each book.
- **Value** = The actual content you get from the selected book.
- **Attention weight** = How relevant each book is to your search.

In Transformers, attention is needed to understand the context and relationships between words.

Example: “The animal didn't cross the road because it was tired.”

- Query: Helps determine what “it” refers to.
- Keys: Help find relevant words, such as “animal.”
- Values: Provide information from those relevant words.

Why needed?

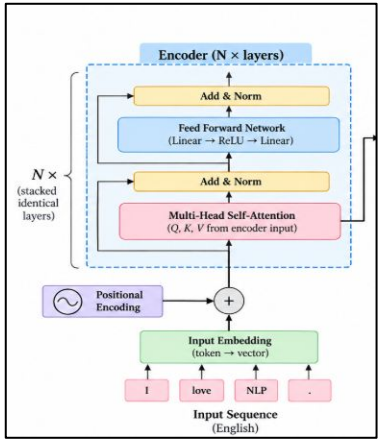
The same word can have different meanings depending on the sentence.

Attention helps each word understand other relevant words and create a **context-aware embedding.**

Step 5: Multi-Head Attention

Why multi-head?

Instead of looking at the sentence in only one way, the Transformer uses several attention heads.



Different heads can learn different relationships:

Head 1 ⟶ grammatical relationship
Head 2 ⟶ word-to-word relationship
Head 3 ⟶ long-distance relationship
Head 4 ⟶ semantic relationship

So:

Multi-Head Attention = Look at the sentence from multiple perspectives.

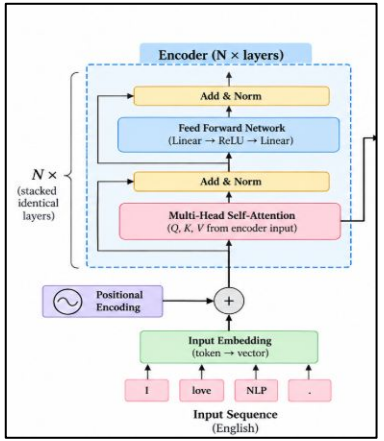
Step 6: Add & Norm

After attention, the result goes through: A**dd + Normalize**

The **Add** part uses a residual/skip connection, helping preserve useful information from earlier in the layer.

Normalize helps keep the values stable during training.

Add & Norm = Helps the network train smoothly and preserve useful information.



Let us understand **Add & Norm in a Transformer using a real sentence.**

Sentence: **"The cat is sitting on the mat."**

Step	Component	Example and Explanation
1	**Input**	The Transformer receives the sentence: "The cat is sitting on the mat." Each word is converted into a numerical vector.
2	**Multi-Head Attention**	The model learns relationships between words. For example, "sitting" may attend to "cat" to understand who is sitting.
3	**Attention Output**	The attention layer generates a new representation containing contextual information about each word.
4	**Add**	The original representation of "cat" is added to its attention output. This preserves the original information while incorporating context.
5	**Norm**	Layer Normalization normalizes the combined numerical values for "cat," helping stabilize training.
6	**Final Output**	The updated representation of "cat" contains both its original information and contextual information from the sentence.

Numerical example of Add & Norm

Suppose we are processing the word **"cat."**

Operation	Numerical Representation
Original input X	$[2, 4]$
Attention output A	$[1, 3]$
Add: $X + A$	$[2 + 1,\ 4 + 3] = [3, 7]$
Norm: Layer Normalization	Normalizes $[3, 7]$ using its mean and variance, with learnable scale and shift parameters.
Output	A normalized representation passed to the next sublayer.

The overall formula is:

$$\boxed{\text{Output} = \text{LayerNorm}(X + A)}$$

Here, X is the original input and A is the attention output.

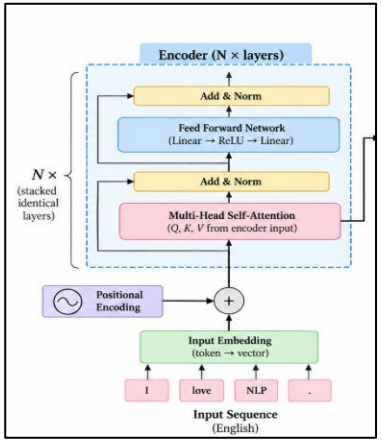
Step 7: Feed-Forward Network

Next, the information goes through a Feed-Forward Neural Network.

Linear
↓
ReLU / activation
↓
Linear

This further processes the information learned by the attention layer.

Feed-Forward Network = Further transforms and learns useful features from each token representation.



Why is it needed?

- Attention learns relationships between words, such as how “cat” relates to “sitting.”
- Feed-Forward Network further processes the representation of each word to learn richer features.

Let's understand it using the same sentence: **"The cat is sitting on the mat."**

Feed-Forward Network: Step-by-step

Step	Component	Example and Explanation
1	**Input**	The output from Add & Norm enters the Feed-Forward Network.
2	**Linear Layer 1**	The network transforms the numerical representation of each word into a larger feature space.
3	**Activation Function (ReLU or GELU)**	Introduces non-linearity, allowing the network to learn complex patterns.
4	**Linear Layer 2**	Transforms the features back to the model's original hidden dimension.
5	**Output**	The updated word representations are passed to the next Add & Norm stage.

Numerical example

Suppose the representation of the word "**cat**" after attention and Add & Norm is:

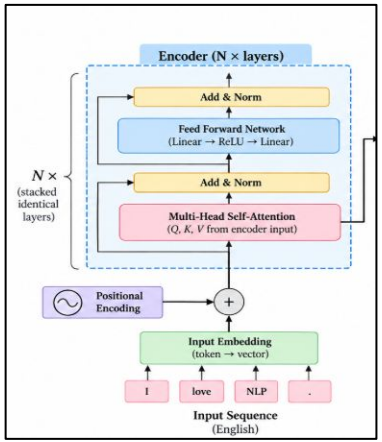
Operation	Numerical Example
Input	$[2, 4]$
First Linear Layer	$[2, 4] \rightarrow [3, 5, 2]$
ReLU Activation	$[3, 5, 2] \rightarrow [3, 5, 2]$
Second Linear Layer	$[3, 5, 2] \rightarrow [4, 6]$
Output	$[4, 6]$

Step 8: Add & Norm Again

After the Feed-Forward Network:

Feed Forward
↓
Add & Norm

Again, the residual connection and normalization help maintain stable learning.



Step 9: Encoder Output

After passing through the Encoder layers, we obtain:

Encoder Output

This contains rich representations of the input sentence.

I ⟶ love ⟶ NLP ⟶ .
↓
Encoder
↓
Rich representation of the input

This information is passed to the Decoder.

DECODER

Step 10: Decoder Input

Suppose we want to translate:

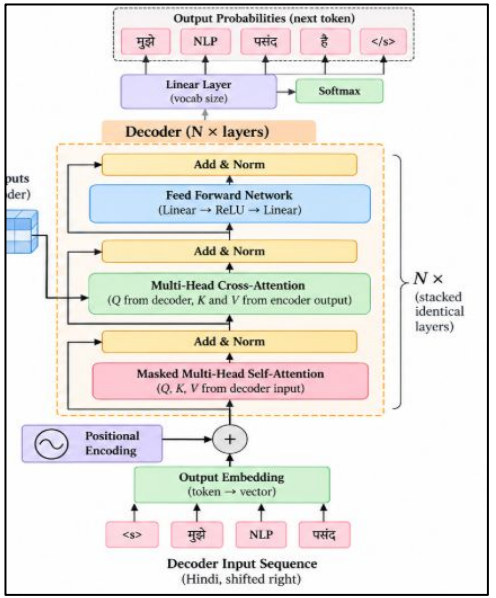
English ⟶ Hindi

The decoder begins with a special token such as:

<s>

Then it generates:

```
Step 1:  <s>
          ↓
Step 2:  मुझे
          ↓
Step 3:  प्राकृतिक
          ↓
Step 4:  भाषा
          ↓
Step 5:  प्रसंस्करण
          ↓
Step 6:  पसंद
          ↓
Step 7:  है
          ↓
Step 8:  </s>
```



Step 11: Masked Self-Attention

The first attention layer in the Decoder is called:

Masked Multi-Head Self-Attention

Why masked?

Because while generating a word, the model should not look at future words.

For example, when predicting:

<s> → मुझे → ?

it can see:

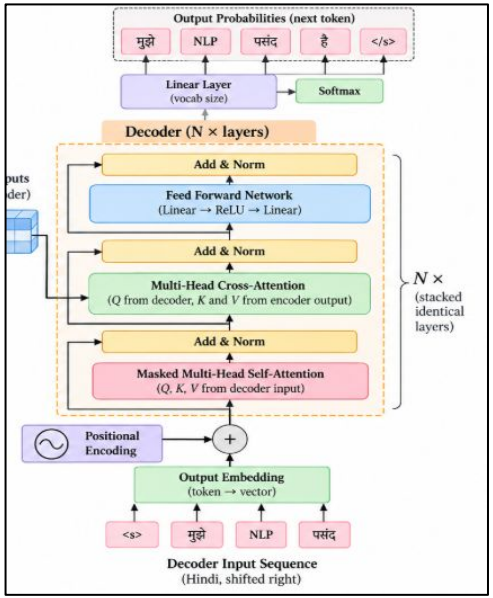
<s>, मुझे

but it cannot see:

प्राकृतिक → भाषा → प्रसंस्करण → पसंद → है →

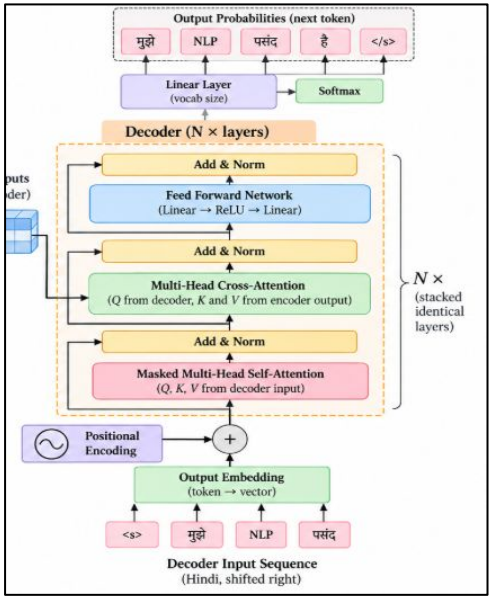
because those words have not been generated yet.

Masking = Prevent the Decoder from cheating by looking at future output words.



Step 12: Cross-Attention

Now comes another very important part: Multi-Head Cross-Attentionallows the decoder to look at the encoder's output while generating the target sentence



So the Decoder asks:

“Which parts of the input sentence should I focus on to generate my next word?”

For example:

English Input
I love NLP
↓
Encoder
↓
Encoder Output
↓
Cross-Attention
↑
Decoder

The Decoder can therefore connect the output it is generating with the relevant information from the input.

Self-Attention:
Looks at relationships within the same sequence.

Cross-Attention:
Connects the Decoder to the Encoder's output.

Cross-Attention: Step-by-step

Step	Component	Explanation using the example
1	**Encoder Output**	The encoder processes “I love NLP” and creates contextual representations of all three words.
2	**Decoder Input**	The decoder receives the Hindi words generated so far, for example, “मुझे”.
3	**Query (Q)**	The decoder creates a Query representing what information it needs to generate the next word.
4	**Key (K)**	The encoder's output provides Keys that help identify relevant input words.
5	**Value (V)**	The encoder's output also provides Values containing contextual information about the input words.
6	**Attention Scores**	The decoder compares its Query with the encoder's Keys to determine which input words are most relevant.
7	**Weighted Sum**	The decoder combines the encoder's Values according to the attention weights.
8	**Output**	The decoder uses this contextual information to help predict the next Hindi word, such as “NLP”.

Difference between Self-Attention and Cross-Attention

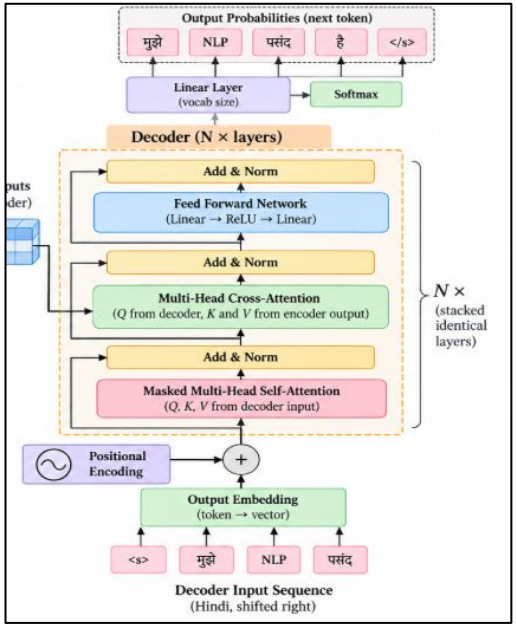
Feature	Self-Attention in Decoder	Cross-Attention in Decoder
Query (Q) comes from	Decoder	Decoder
Key (K) comes from	Decoder	Encoder
Value (V) comes from	Decoder	Encoder
Main purpose	Understand relationships among generated words	Connect generated words with the input sentence
Example	Understand “मुझे” in the generated Hindi sequence	Refer to “I love NLP” to help predict the next Hindi word

Step 13: Feed-Forward Network

After cross-attention, the information again passes through a: Feed-Forward Network

This further processes the representation.

Then another: Add & Norm is applied.



OUTPUT

Step 14: Linear Layer

Now the Decoder has a representation of what it wants to generate.

But we still need to choose an actual word.

The Linear Layer converts the representation into scores for every word in the vocabulary.

Token	Softmax Probability
प्रसंस्करण	**0.72**
पसंद	0.12
सीख	0.06
भाषा	0.04
अन्य शब्द	0.06

Softmax during Transformer Translation

Suppose the Decoder has generated:

<s> → मुझे → प्राकृतिक → भाषा

Now it needs to predict the next token.

The Decoder produces scores for all possible Hindi tokens in its vocabulary.

For example:

Token	Softmax Probability
प्रसंस्करण	**0.72**
पसंद	0.12
सीख	0.06
भाषा	0.04
अन्य शब्द	0.06

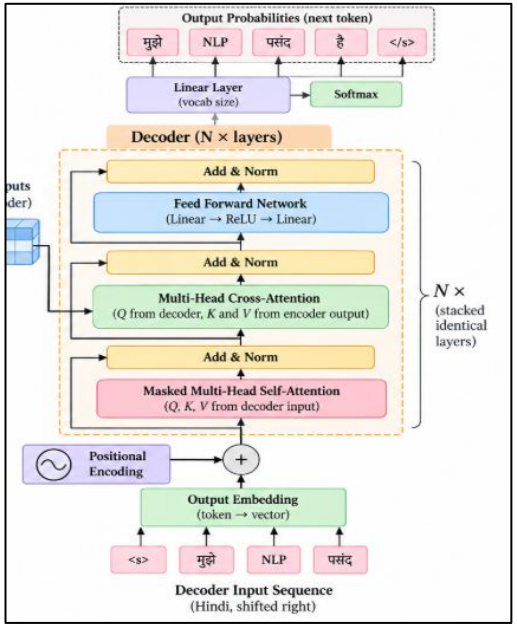
Softmax converts the Decoder's raw scores into probabilities.

Since “प्रसंस्करण” has the highest probability, it is selected as the next token:

<s> → मुझे → प्राकृतिक → भाषा → प्रसंस्करण

Then the process repeats:

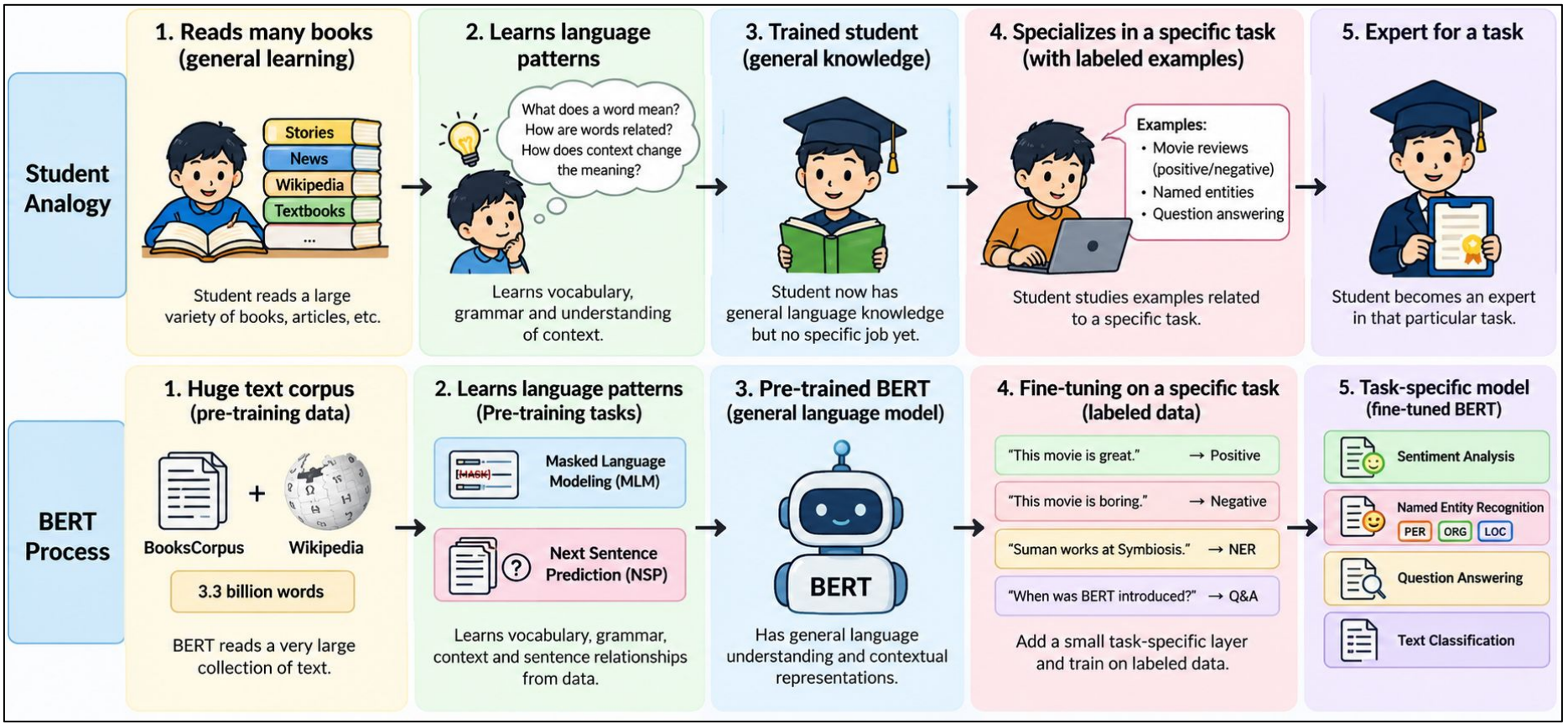
```
Previous tokens
      ↓
   Decoder
      ↓
   Linear Layer
      ↓
    Softmax
      ↓
Probability of every token
      ↓
Highest-probability token
      ↓
Next token
```



BERT

Bidirectional Encoder Representations from Transformers

“Understanding BERT Through a Student Learning Analogy



Fine-tuning era : BERT

BERT is a language understanding model that reads words by looking at both the left and right context.

For example:

Sentence:

I went to the bank to deposit money.

The word bank can have different meanings:

- Bank → financial institution
- Bank → riverbank

BERT looks at the complete context:

I went to the BANK to deposit MONEY

←———————— **BANK** ————————→

left context | right context

Because BERT sees "deposit money", it understands that bank = financial institution. This is the central idea of BERT.

BERT's core idea

Pre-train deeply bidirectional

Train BERT on a large amount of unlabeled text.

It learns language by:

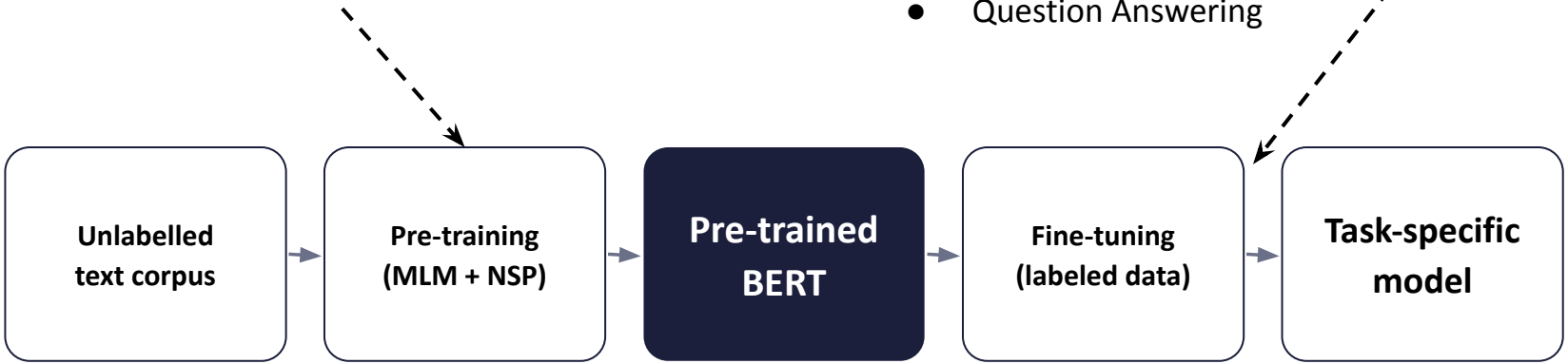
- Predicting missing (masked) words
- Understanding the relationship between sentences

Fine-tune for any task

After pre-training, use the same BERT model for a specific task.

Add a small output layer and train it with **labeled data** for tasks such as:

- Text classification
- Named Entity Recognition (NER)
- Question Answering



1. Unlabeled Data — During Pre-training

Data has **text but no answer/label**.

Example:

> **"The cat is sitting on the mat."**

BERT hides a word:

> **"The cat is [MASK] on the mat."**

BERT tries to predict "**sitting**".

So, **unlabeled data = text without predefined answers**.

2. Labeled Data — During Fine-tuning

Data contains **text + the correct answer/label**.

Example for sentiment classification:

Text	Label
"This movie is excellent."	Positive
"This movie is boring."	Negative

BERT learns from these examples to classify new sentences.

Unlabeled data:

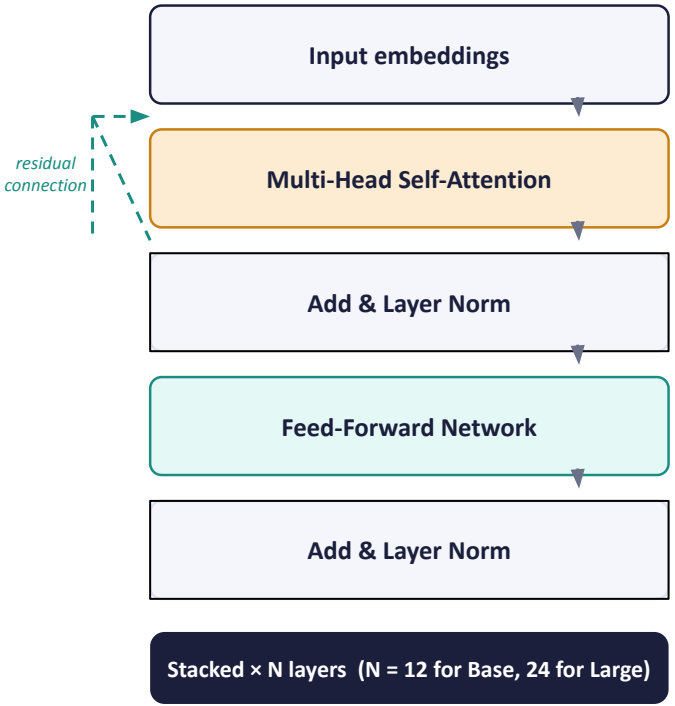
> Text → **No answer given**

Labeled data:

> Text → **Correct answer/label given**

BERT:
Unlabeled text → Pre-training → Labeled data → Fine-tuning → Specific task

BERT -Building block: the Transformer encoder



Every token computes a Query, Key, and Value vector.

Self-Attention —Each word looks at other words in the sentence to understand its meaning.

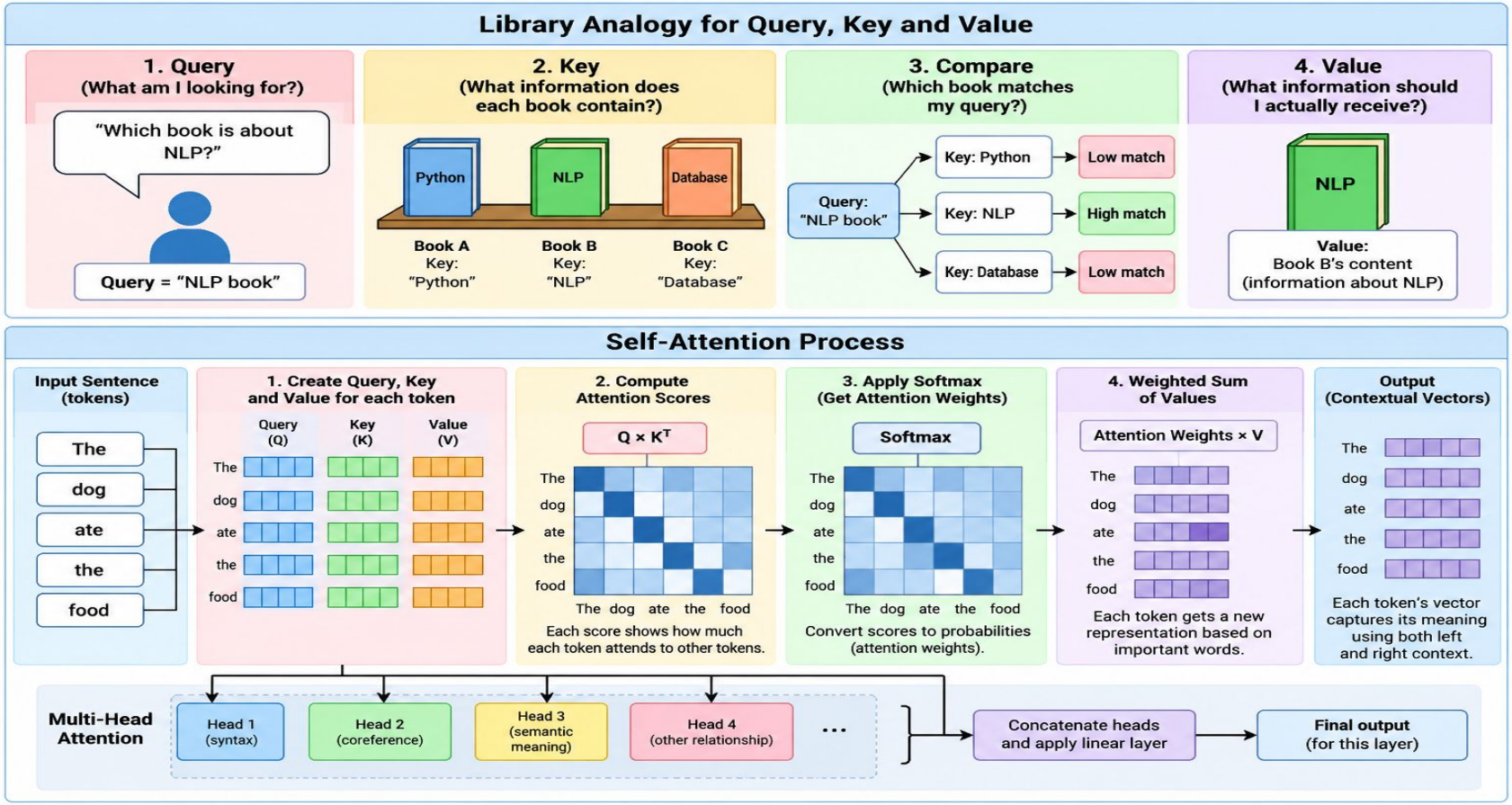
- It checks which words are important and how strongly they are related.
- It can look at words before and after it — both left and right.
- Different attention heads learn different relationships, such as:
  - **Grammar**
  - **Word meaning**
  - **Pronoun/reference relationships**
  - **Relationships between words**
- This is why **BERT is called bidirectional:** it understands a word using both its left and right context.

Sentence:

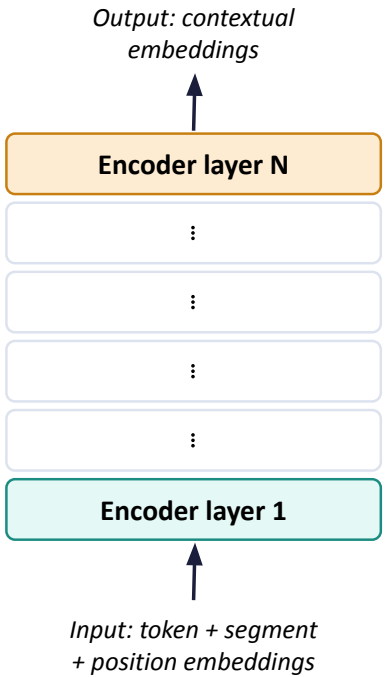
The dog ate the food because it was hungry.

When BERT processes “it”, it can look at “dog” and other words around it to understand what “it” refers to.

Every token computes a Query, Key, and Value vector.



BERT = a stack of Transformer encoders

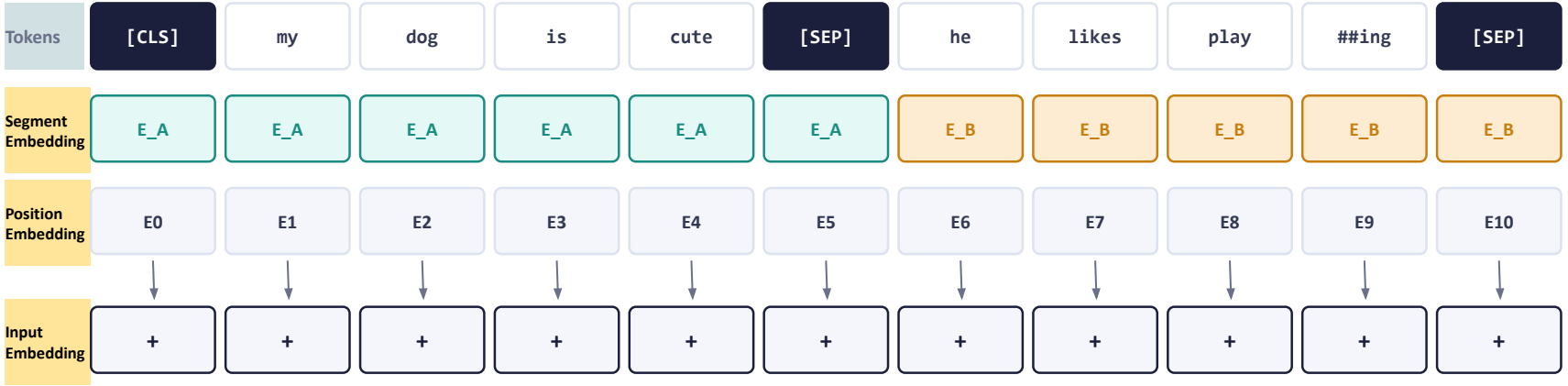


Model	Layers (N)	Hidden size	Attn. heads	Parameters
BERT-Base	12	768	12	110M
BERT-Large	24	1024	16	340M

- Each encoder layer uses self-attention to understand relationships between **words and a neural network to process the information.**
- BERT can process up to 512 tokens at a time and understands the position of each token.
- Each token gets a numerical representation (vector) of size 768 or 1024.
- The same word can have different representations depending on the sentence.

Example: “**bank**” in river bank and **bank account** gets different representations because the meanings are different.

How a sentence becomes BERT's input



Why three embeddings?

BERT gives each word/sub-word three types of information:

- **Token Embedding:** Tells BERT what the word is.
  Example: playing → play + ##ing
- **Segment Embedding:** Tells BERT which sentence the word belongs to.
  Example: Sentence A or Sentence B.
- **Position Embedding:** Tells BERT where the word appears in the sentence.
  Example: 1st word, 2nd word, 3rd word, etc.

Why ##ing?

BERT breaks uncommon words into smaller known pieces.

Example:

playing → play + ##ing

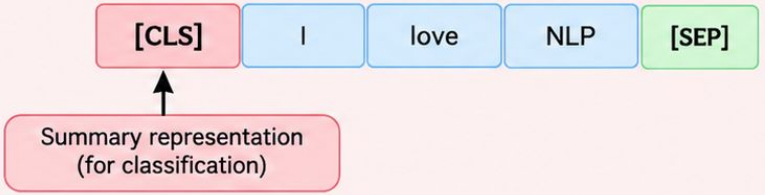
So, BERT can understand new or rare words using smaller word piece

A summary representation is a single vector that captures the overall meaning of the entire sentence or text.

BERT Special Tokens

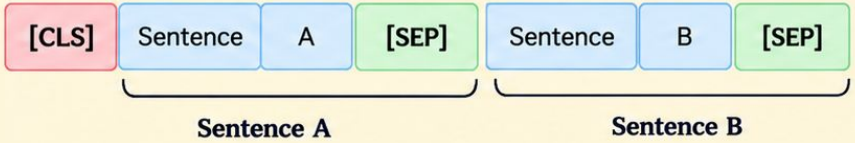
1. [CLS] – Classification token

- Placed at the beginning of the input.
- For classification, BERT uses the final representation of [CLS] as the summary of the whole sentence.



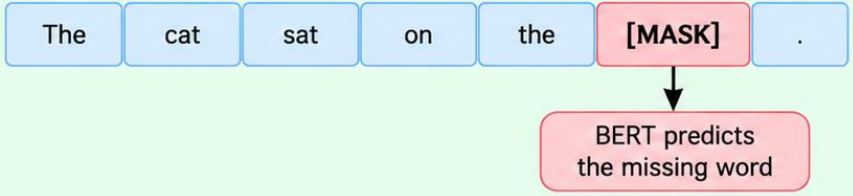
2. [SEP] – Separator token

- Separates sentences.
- Used for sentence-pair tasks.



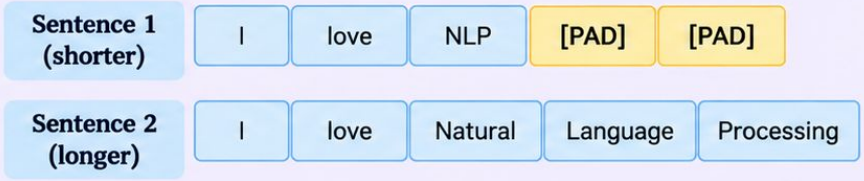
3. [MASK] – Mask token

- Used during pre-training.
- BERT has to predict the missing word.



4. [PAD] – Padding token

- Used when sentences have different lengths.
- Added to the shorter sentence to make all inputs the same length.



Masked Language Modeling (MLM) in BERT

1. Original Sentence

Input sentence:

The	cat	sat	on	the	mat	.

2. Masked Input to BERT

Some tokens are randomly selected and replaced. Here, the last word is masked.

The	cat	sat	on	the	[MASK]	.

3. BERT Predicts the Missing Word

The	cat	sat	on	the	[MASK]	.

BERT
(Pre-trained Model)

Predicted probability distribution

Possible Words	Probability
mat	62%
floor	14%
couch	9%
...	...

4. Example with Different Context

Input sentence:

The	children	went	to	the	playground	and	played	on	the	[MASK]	.

BERT
(Pre-trained Model)

Possible Words	Probability
swing	58%
slide	21%
car	8%
computer	4%
...	...

5. 80/10/10 Rule for Creating Masked Input

When selecting tokens for masking during pre-training, BERT uses the following strategy:

80% → Replace with [MASK]

Most of the time, the token is replaced with **[MASK]**.

The	cat	sat	on	the	[MASK]	.

80% of the time

10% → Replace with a Random Word

Sometimes, the token is replaced with a random word.

The	cat	sat	on	the	dog	.

10% of the time

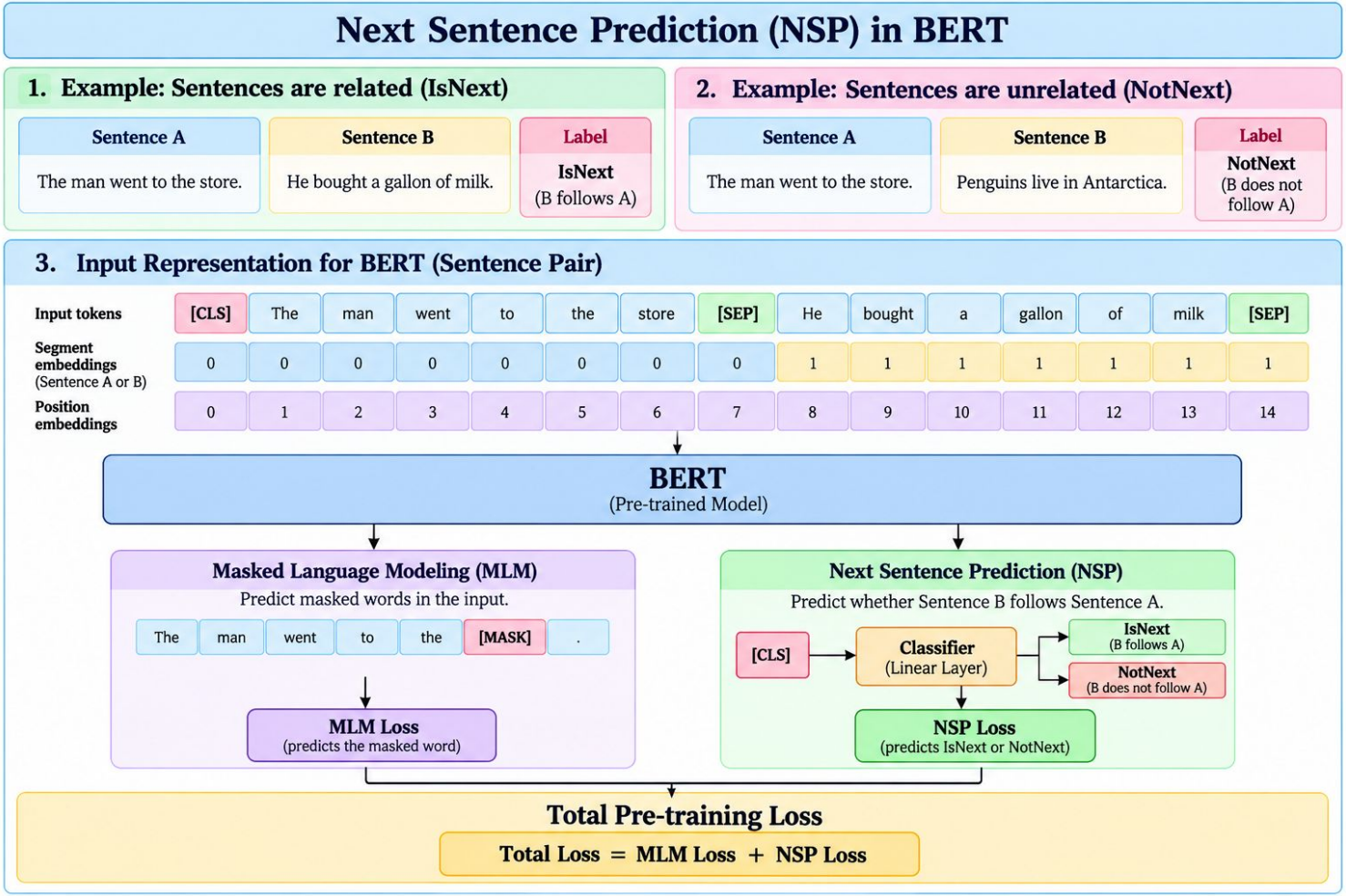
10% → Keep the Original Word

Sometimes, the token is kept as the original word (without any change).

The	cat	sat	on	the	mat	.

10% of the time

Why this rule? During real-world use, BERT does not usually receive [MASK] tokens. This strategy helps reduce the mismatch between pre-training (where [MASK] is used) and later use (where no [MASK] is used).


Next Sentence Prediction (NSP) in BERT
1. Example: Sentences are related (IsNext)
Sentence A
The man went to the store.
Sentence B
He bought a gallon of milk.
Label
IsNext
(B follows A)
2. Example: Sentences are unrelated (NotNext)
Sentence A
The man went to the store.
Sentence B
Penguins live in Antarctica.
Label
NotNext
(B does not follow A)
3. Input Representation for BERT (Sentence Pair)
Input tokens
[CLS] The man went to the store [SEP] He bought a gallon of milk [SEP]
Segment embeddings
(Sentence A or B)
0 0 0 0 0 0 0 0 1 1 1 1 1 1 1
Position embeddings
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14
BERT
(Pre-trained Model)
Masked Language Modeling (MLM)
Predict masked words in the input.
The man went to the [MASK] .
MLM Loss
(predicts the masked word)
Next Sentence Prediction (NSP)
Predict whether Sentence B follows Sentence A.
[CLS]
Classifier
(Linear Layer)
IsNext
(B follows A)
NotNext
(B does not follow A)
NSP Loss
(predicts IsNext or NotNext)
Total Pre-training Loss
Total Loss = MLM Loss + NSP Loss

BERT Pre-training Process

1. Large Unlabeled Text Data

3.3 billion words



BooksCorpus
(~800 million words)



English Wikipedia
(~2.5 billion words)

Total 3.3 billion words

2. BERT Learns from the Text (Pre-training)



Reads billions of words

↓

Practices missing words (Masked Language Modeling)

↓

Learns sentence relationships (Next Sentence Prediction)

↓

Learns language

3. Pre-training Tasks

Masked Language Modeling (MLM)

The cat sat on the **[MASK]** .

Predict the missing word using context.

Next Sentence Prediction (NSP)

A: The man went to the store.
B: He bought a gallon of milk. → **IsNext** (B follows A)

A: The man went to the store.
B: Penguins live in Antarctica. → **NotNext** (B does not follow A)

4. Pre-trained BERT Model

BERT
(Pre-trained Model)

↓

Learns general language understanding (from unlabeled text)

↓

Can be fine-tuned for different NLP tasks (text classification, NER, question answering, etc.)

5. Pre-training Settings and Training Duration (Original BERT)

a) Maximum Sequence Length

[CLS] w_1 w_2 ⋯ w_{511} [SEP]

Maximum sequence = 512 tokens

b) Batch Size



Batch = 256 sequences
(256 sentences at a time)

c) Training Steps



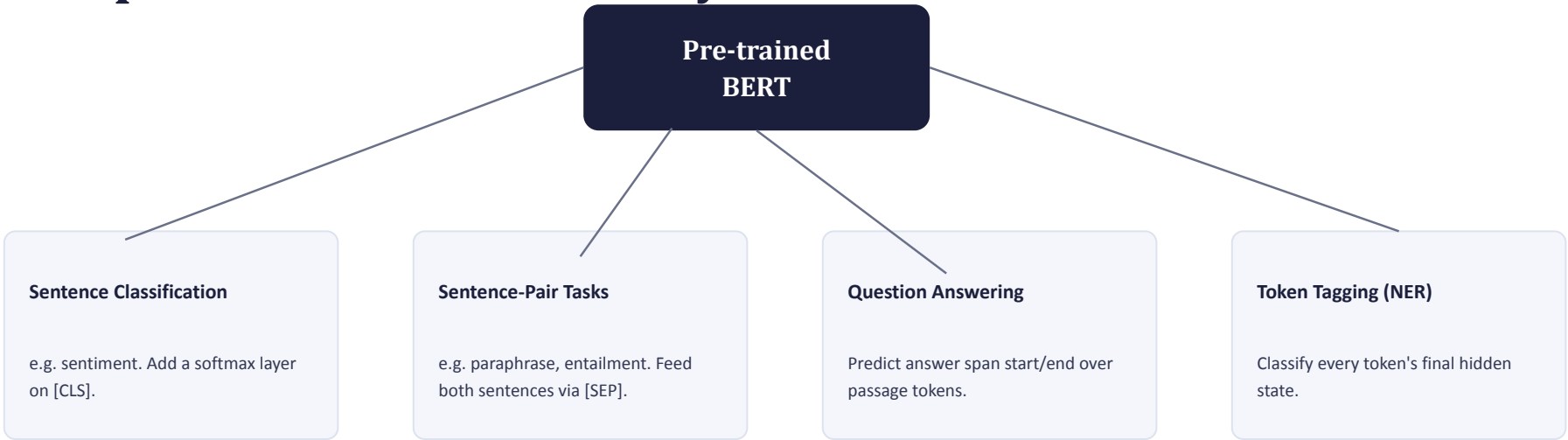
Training = 1 million steps
(1,000,000 steps)

d) Training Duration (Original Paper)

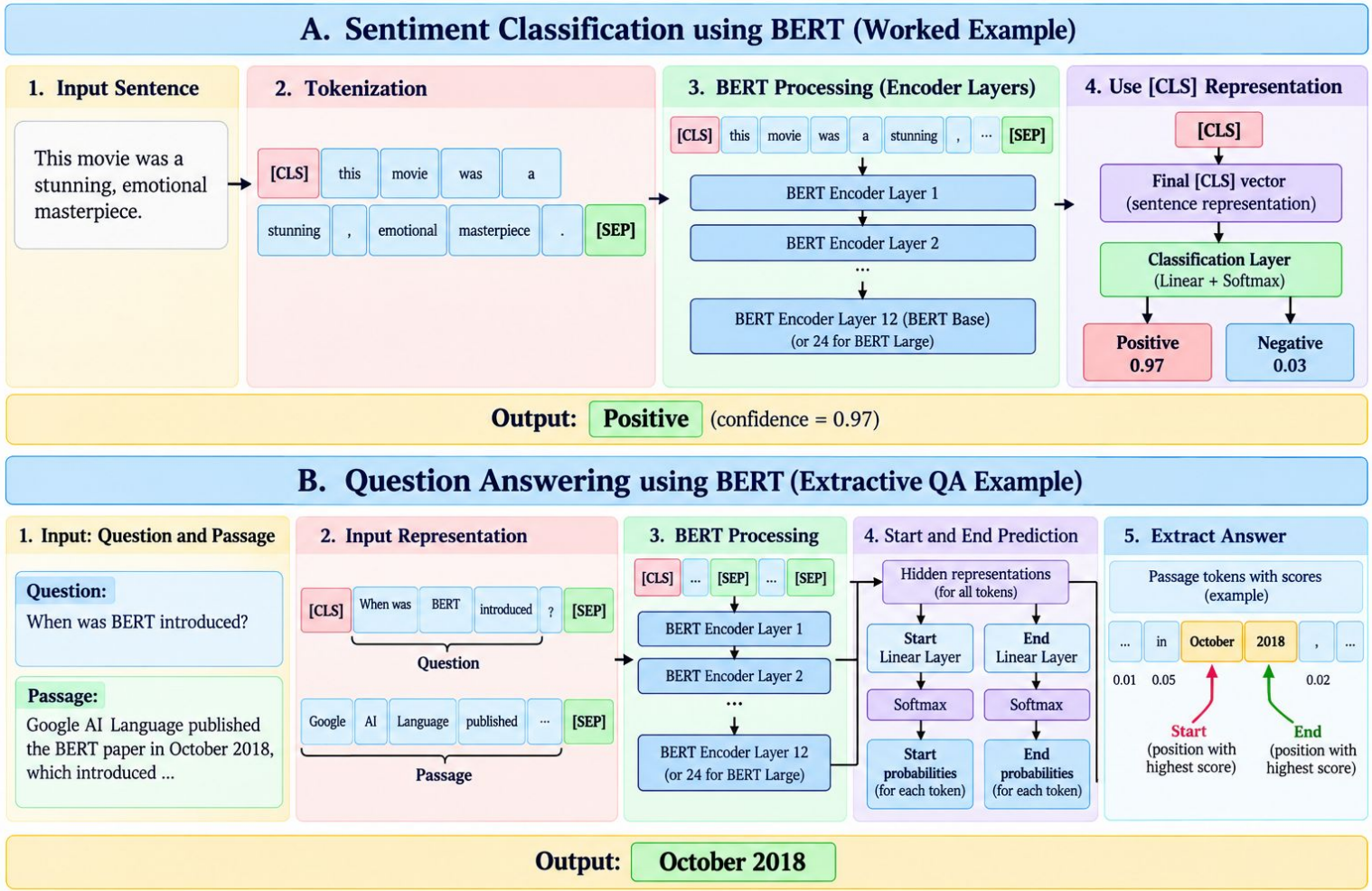
Model	Parameters	TPU Training Time
BERT Base	110 million	~ 4 days
BERT Large	340 million	~ 16 days

Result: A pre-trained BERT model that understands general language, which can be fine-tuned for many NLP tasks.

One pre-trained model, many fine-tuned tasks



Only the top layer changes. Every task reuses the same 12/24-layer encoder and its weights as initialization; fine-tuning typically needs only a few epochs and a small learning rate (2e-5 to 5e-5) on a single GPU — a fraction of pre-training's cost.


A. Sentiment Classification using BERT (Worked Example)
1. Input Sentence
This movie was a stunning, emotional masterpiece.
2. Tokenization
[CLS] this movie was a stunning , emotional masterpiece . [SEP]
3. BERT Processing (Encoder Layers)
[CLS] this movie was a stunning , ... [SEP]
BERT Encoder Layer 1
BERT Encoder Layer 2
...
BERT Encoder Layer 12 (BERT Base)
(or 24 for BERT Large)
4. Use [CLS] Representation
[CLS]
Final [CLS] vector
(sentence representation)
Classification Layer
(Linear + Softmax)
Positive
0.97
Negative
0.03
Output: Positive (confidence = 0.97)
B. Question Answering using BERT (Extractive QA Example)
1. Input: Question and Passage
Question:
When was BERT introduced?
Passage:
Google AI Language published the BERT paper in October 2018, which introduced ...
2. Input Representation
[CLS] When was BERT introduced ? [SEP]
Question
Google AI Language published ... [SEP]
Passage
3. BERT Processing
[CLS] ... [SEP] ... [SEP]
BERT Encoder Layer 1
BERT Encoder Layer 2
...
BERT Encoder Layer 12
(or 24 for BERT Large)
4. Start and End Prediction
Hidden representations
(for all tokens)
Start
Linear Layer
End
Linear Layer
Softmax
Softmax
Start
probabilities
(for each token)
End
probabilities
(for each token)
5. Extract Answer
Passage tokens with scores
(example)
... in October 2018 , ...
0.01 0.05 0.02
Start
(position with
highest score)
End
(position with
highest score)
Output: October 2018

BERT Family

BERT	Bidirectional Transformer	Understands words using context from both sides	**Sentiment analysis:** "This phone is amazing" → Positive
RoBERTa	More data + longer training; removes NSP	A stronger version of BERT trained more extensively	**News classification:** Classify an article as sports, politics, business, etc.
DistilBERT	Smaller distilled version	A smaller and faster BERT that keeps much of BERT's ability	**Chatbot on a mobile device:** Quickly classify customer questions
ALBERT	Parameter sharing + factorized embeddings	Reduces the number of parameters to make BERT more efficient	**Large-scale text classification:** Process many customer reviews with less memory
ELECTRA	Replaced-token detection	Instead of predicting a missing word, it detects whether a word has been replaced	**Text quality checking:** Detect unusual or incorrect words in a sentence
DeBERTa	Disentangled attention	Separately considers **word content** and **word position** to understand relationships better	**Question answering:** Find the correct answer from a long passage

From Fine-tuning to Prompting

It means moving from training the model for a specific task to giving instructions to an already-trained model.

Fine-tuning: Train BERT with labeled data → Model learns a specific task.

Prompting: Give the model a clear instruction → Model performs the task without changing its parameters.

Example

Fine-tuning: Train BERT on thousands of movie reviews → BERT learns sentiment classification.

Prompting:

Give an AI: "Classify this review as Positive or Negative: *This movie was amazing.*" → Positive

Means:

Fine-tuning = Teach the model.

Prompting = Tell the model what to do.

Generative Pre-trained Transformer (GPT)

What is GPT?

GPT is a language model that reads and generates human-like text.

It learns statistical patterns and relationships between words from massive datasets, then reuses that single learned mode across many different language tasks.

Meaning of GPT

Term	Simple meaning	Example
G – Generative	Creates new content	Writes an email or a story
P – Pre-trained	Learns language patterns before being used for a specific task	Learns from large amounts of text
T – Transformer	A neural network architecture that understands relationships between words using attention	Understands the meaning of a word based on its sentence

A real-life example

Suppose you type:

“The sun rises in the _____.”

GPT uses the context and its learned patterns to predict the likely next word: East.

This is a simplified example of next-token prediction, one of the key ideas behind GPT.

Remember:

- GPT is a language model based on the Transformer architecture.
- It generates responses by predicting tokens based on the input and the context it has processed.
- It can sometimes make mistakes, so its answers are not always correct.

How GPT Works: Two Main Pillars

GPT works by combining Generative Pre-training and the Transformer Architecture. One explains how GPT learns, and the other explains how it processes language.

1. Generative Pre-training

Meaning: How GPT learns language

GPT learns patterns from large amounts of text before being used to answer questions or perform specific tasks.

2. Transformer Architecture

Meaning: How GPT processes language

The Transformer uses self-attention to identify relationships between words and understand their context.

GENERATIVE PRE-TRAINING

Learning like a child who reads a lot

The child

Reads thousands of storybooks. Nobody teaches grammar rules, yet they learn which words usually come next.

GPT

Reads a huge amount of text and practises one task again and again: **predict the next word.**

Example: filling in the blank

The sun rises in the _____



Repeated across billions of sentences, this builds up grammar, facts, writing styles and reasoning patterns.

GENERATIVE PRE-TRAINING

"Generative": writing one word at a time

"Write a short poem about rain."

GPT picks the most suitable next word, adds it, then predicts again.

Rain →

Rain falls →

Rain falls softly →

Rain falls softly on →

Rain falls softly on the roof…

Self-attention: what does "it" mean?



High attention Word being read

Looks at every word

While reading "it", it weighs how much each other word matters.

"cat" + "tired"

Mats don't get tired, so "it" means the cat.

Change one word, change the meaning



"mat" + "soft"

Now "it" means the mat, because mats can be soft.

Context decides meaning

The same word gets a different meaning depending on the words around it.

BOTH PILLARS TOGETHER

"Who wrote Romeo and Juliet?"

STEP 1

You ask

The question is broken into words for the model to read.



STEP 2 · TRANSFORMER

Understands

Self-attention links "who wrote" (an author) with "Romeo and Juliet" (a play).



STEP 3 · PRE-TRAINING

Recalls

Uses patterns learned from text where this fact appeared many times.

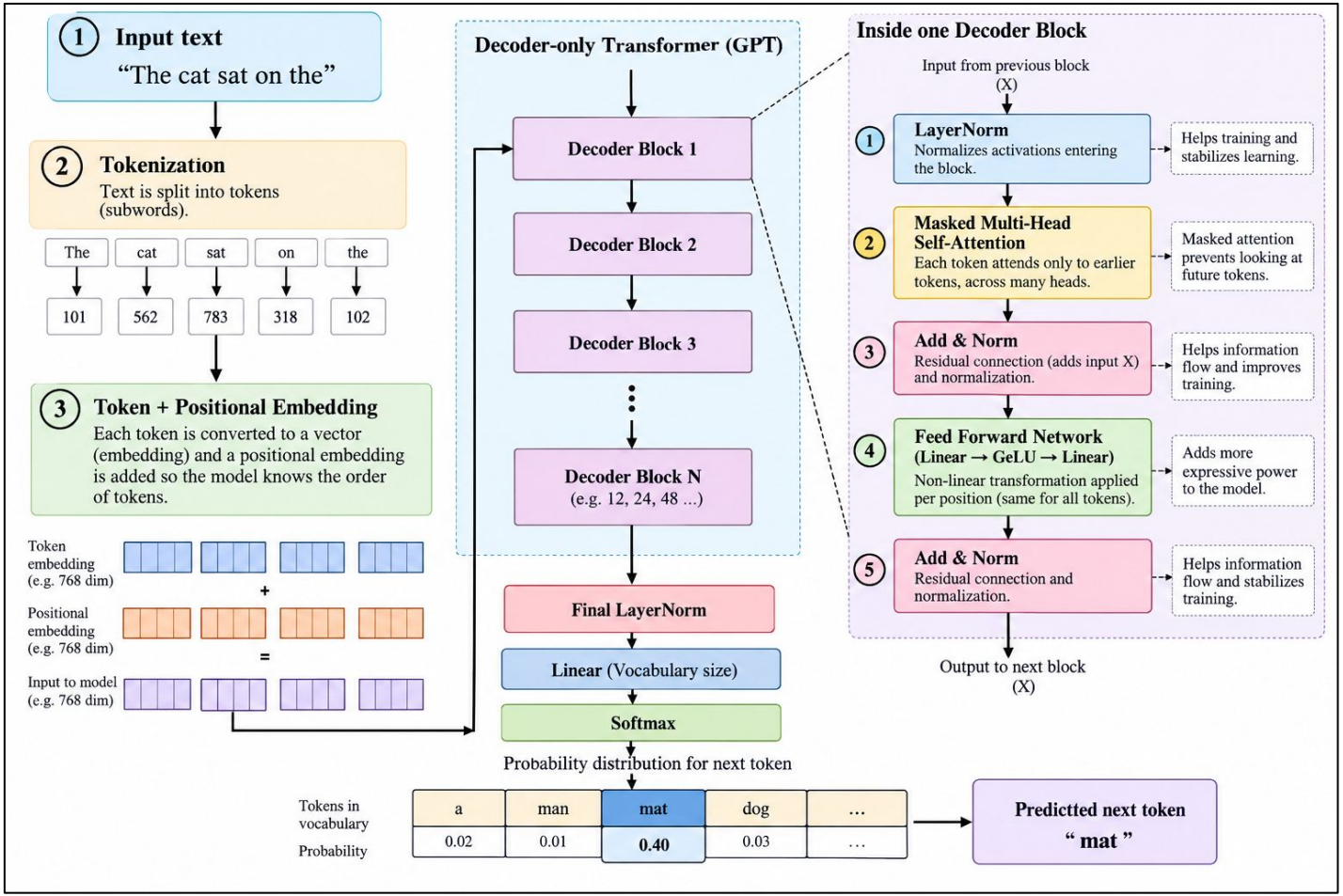


STEP 4 · GENERATE

Answers

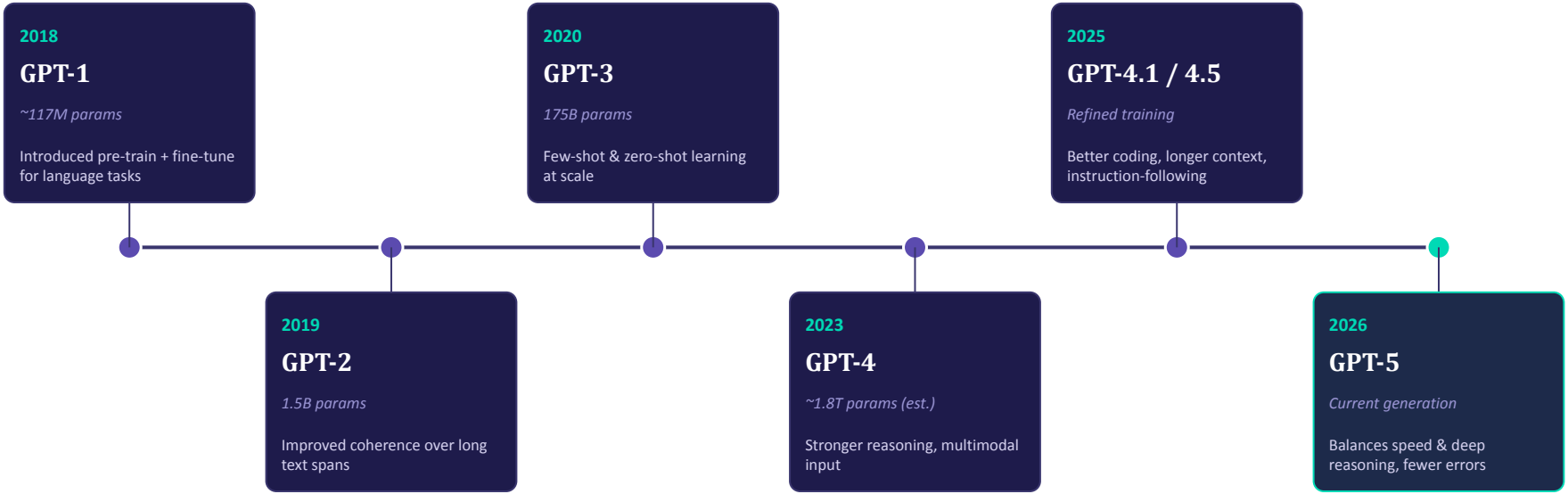
Word by word: "William Shakespeare wrote Romeo and Juliet."

ARCHITECTURE



From GPT-1 to GPT-5

Each generation scaled parameters and training methods, unlocking new capabilities.



GPT vs. BERT: two transformer philosophies

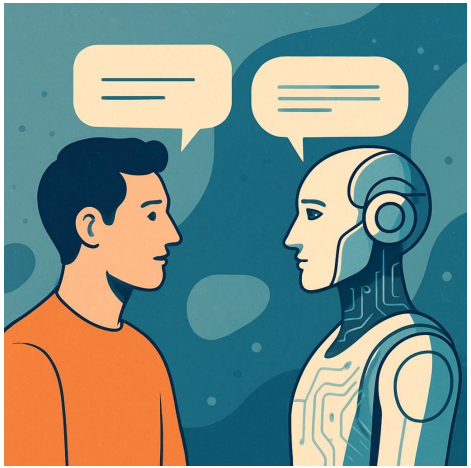
Both are built on the transformer, but they use opposite halves of it for opposite goals.

	GPT (Generative)	BERT (Understanding)
Architecture half	Decoder-only	Encoder-only
Reads context	Left-to-right (causal / masked)	Bidirectional (whole sentence at once)
Primary goal	Generate the next token	Understand & represent existing text
Great at	Writing, chat, completion, coding	Classification, search, sentence similarity
Typical use	ChatGPT-style assistants	Search ranking, sentiment analysis

Conversational AI

Conversational AI is a technology that allows computers to understand and communicate with humans using natural language.

Human → Question → AI understands → AI processes → Response → Conversation



human + chatbot/robot with speech bubble



How does Conversational AI work?



1. User Input
Human asks or types a question.

"What is BERT?"



2. Understand
AI understands the question (natural language understanding).



3. Process
AI finds relevant information or generates a response (using a language model).



4. Generate Response
AI gives a suitable answer in natural language.

"BERT is a language model based on Transformers."

Real-world Examples



Chatbots
Customer support (FAQs, issue resolution)



ChatGPT
Answering questions, having conversations



Siri / Alexa
Voice-based interaction



Banking chatbots
Checking account information, transactions

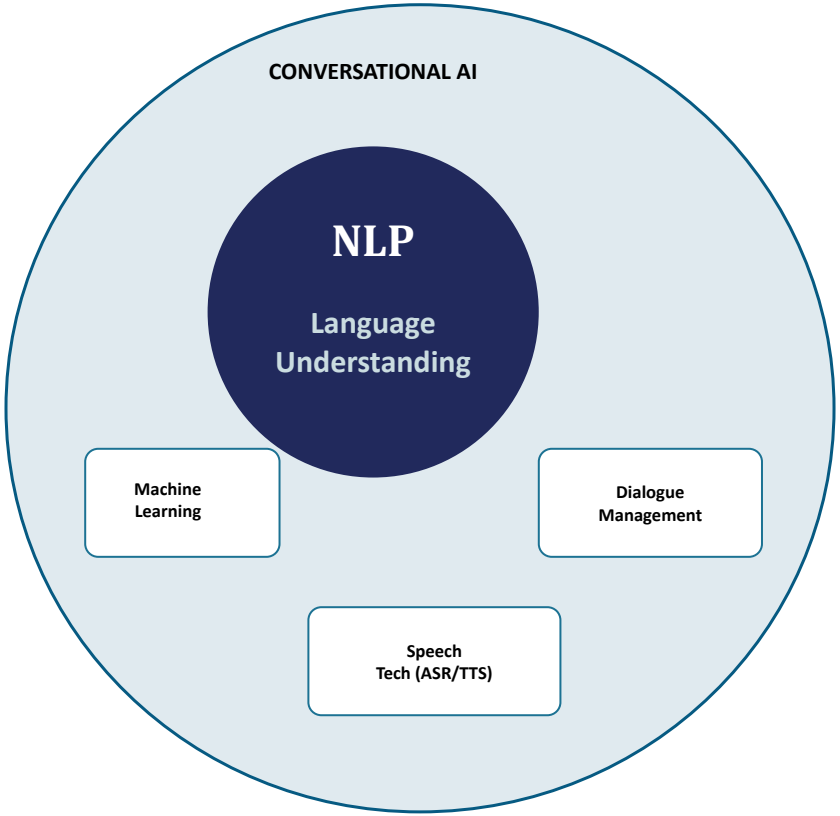


Shopping assistants
Helping select products, recommendations

What Is Conversational AI?

Conversational AI is the **larger system** built to actually interact with people.
It combines NLP with machine learning, dialogue management, natural language generation, and often speech recognition or text-to-speech.

Conversational AI helps machines talk with people.



EXAMPLES

- Virtual assistants (Siri, Alexa)
- Customer support chatbots
- Voice bots for contact centers
- Task-guiding AI assistants

Difference between NLP and Coversational AI

Area	NLP	Conversational AI
What it is	A branch of AI focused on language understanding & generation	A larger system designed to interact with users in natural conversation
Main goal	Process and interpret human language	Manage back-and-forth conversations
Output	Insights, labels, extracted meaning, or generated text	Context-aware replies and actions
Context handling	Usually limited to a single input	Maintains context across multiple turns
Core use	Sentiment analysis, translation, classification	Virtual assistants, chatbots, voice agents

How Conversational AI Uses NLP

NLP is one stage in a longer pipeline — conversational AI wraps it with decision-making and delivery.



Without NLP, conversational AI cannot understand language well. Without dialogue management and response orchestration, NLP alone cannot hold a useful conversation — each stage depends on the ones around it.

SECTION 05 — WORKED EXAMPLE
Tracing a Real Support Call

CUSTOMER SAYS



"Hi, I'd like to check the status of my order, number 12345."

Speech-to-Text

Audio is converted into text the system can read.

STT

Intent Recognition

Identifies the goal: "check order status."

Intent

Entity Extraction

12345

Pulls out the key value: order number "12345."

NER

Database Search

System queries internal records using that ID.

Lookup

Response Generation

Forms a natural reply from the result.

NLG

AI RESPONDS



"Order 12345 is being prepared for shipment. It's expected to arrive by Friday."

---

1. Customer Voice Input

(Audio)

2. Convert to Text
(Speech-to-Text)

"Hi, I'd like to check the status of my order, number 12345."

(Text)

3. Intent Detection

Intent: check order status

4. Entity Extraction

Order number: 12345

5. Database Search

Query: order_id = 12345

Order Database

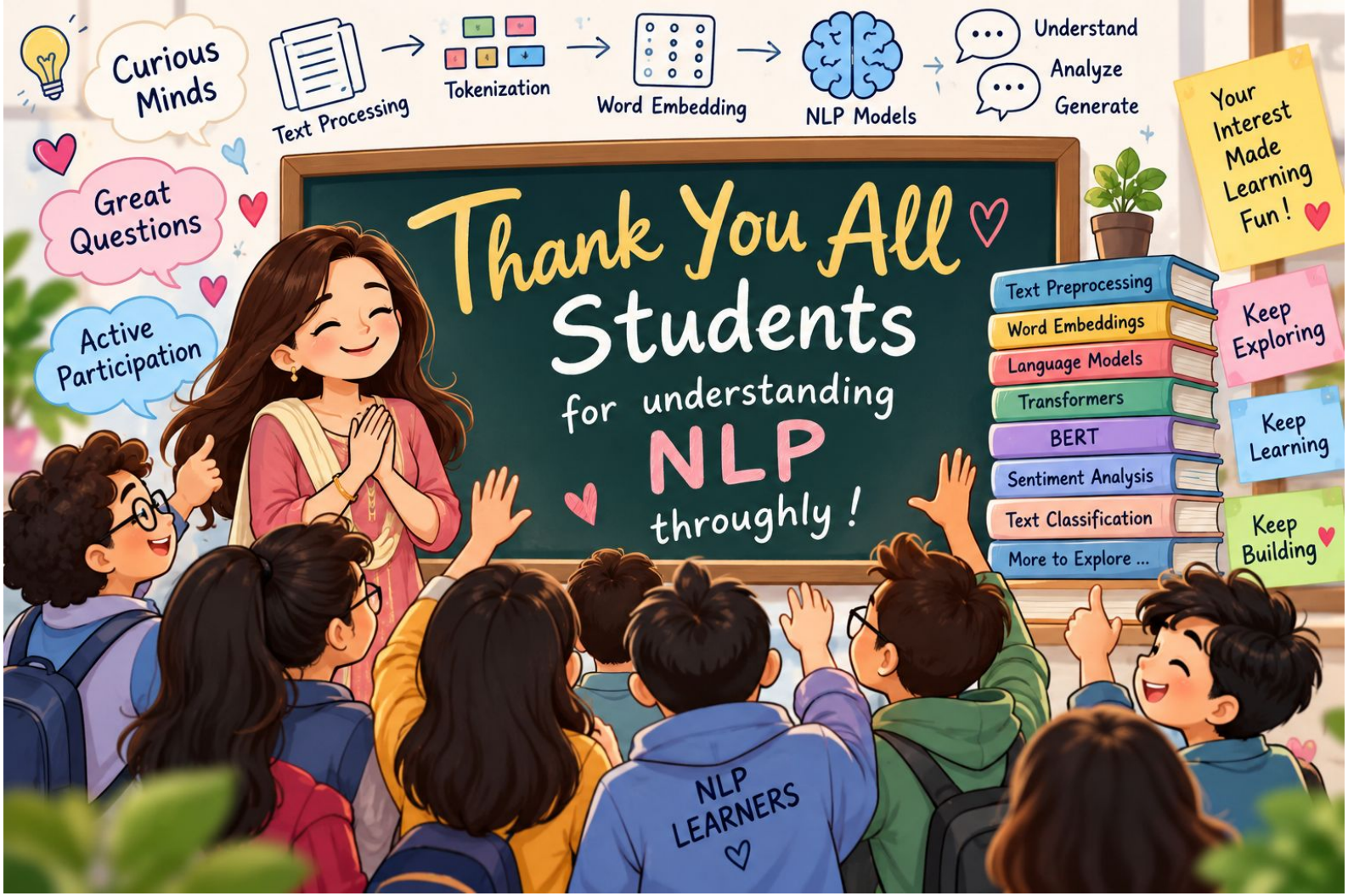
Find order details for 12345

6. Generate Response

Order 12345 is being prepared for shipment. It's expected to arrive by Friday.

WHY THIS MATTERS

- Every step above is powered by an NLP technique — but none of them, alone, *produces the final spoken reply*.
- Dialogue orchestration ties NLP's outputs to a database action, then hands off to language generation.
- The entire sequence runs in milliseconds, end to end, on a single customer turn.


Curious Minds
Text Processing
Tokenization
Word Embedding
NLP Models
Understand
Analyze
Generate
Great Questions
Active Participation
Thank You All
Students
for understanding
NLP
throughly !
Your Interest Made Learning Fun !
Text Preprocessing
Word Embeddings
Language Models
Transformers
BERT
Sentiment Analysis
Text Classification
More to Explore ...
Keep Exploring
Keep Learning
Keep Building
NLP LEARNERS